

OC CS Speckit — Rules & Specifications

Generated from `.cursor/rules/`, `docs/adr/`, `docs/nfr/`, `docs/arch_diagrams/`, and `features/` (auto-discovered). Mermaid fences (including C4) are rendered in the PDF via `md-mermaid-pdf`.

Part 1: Cursor Rules

constitution.mdc

The Global Project Constitution

This document establishes the absolute operational guardrails for this engineering project. All team members and AI execution loops must respect these principles. Any code generation that violates this constitution must be rejected immediately.

Principle 1: Spec-Driven Absolute Truth

- **The Law:** No application code may be generated, modified, or refactored unless there is a corresponding, explicit requirement written in a Markdown file inside the `features/` directory.
- **The Guardrail:** If a developer prompts you to build a feature, you must first ask: "Where is the feature markdown file specification?" If it does not exist, refuse to generate the code.

Principle 2: Branching & Git Discipline

- **The Law:** You are strictly forbidden from writing or pushing feature implementation directly to the `main` or `master` branches. `main` must remain the scaffold-only starter kit (specs, rules, empty app shell, test harness).
- **The Guardrail:** Create a `dev` branch from `main` for integration. All feature development takes place on dedicated branches named `feature/[N]-[feature-name]` (e.g., `feature/1-user-auth`), branched from `dev` and merged back into `dev` when complete. Never merge feature branches into `main`. Operational gate for agents: [feature-branch.mdc](#).

Principle 3: No Ghost Testing / Strict Verification

- **The Law:** Code cannot be considered "Done" until a matching automated test suite passes.
- **The Guardrail:** You must never write a feature file block without also updating or generating its matching test verification parameters. You are forbidden from writing tests that merely assert `true === true`; tests must actively attempt to break the code using edge-case inputs.

Principle 4: Atomic Commit Generation

- **The Law:** Do not execute massive, sweeping changes across dozens of files at the same time.
- **The Guardrail:** Work iteratively. Implement changes in micro-steps (e.g., Sequelize model in `backend/app/models/`, then route + controller, then `frontend/src/services/`, then view). Follow the split `frontend/` + `backend/` layout defined in `project-structure.mdc`.

Principle 5: Stack Consistency

- **The Law:** Implementation must follow the patterns documented in `.cursor/rules/` unless a feature spec explicitly documents a deviation.

- **The Guardrail:** Vue 3 + Vuetify 4 + axios on the frontend; Node + Express + Sequelize (ES modules) on the backend. Do not introduce alternate frameworks, response envelopes, or directory layouts without updating the spec and rules first.



Principle 6: The Developer is the Tech Lead

- **The Law:** The AI is the executor; the human developer is the Architect and Auditor.
- **The Guardrail:** You must explain *why* you chose a specific logical architecture block if asked. Do not hide complex code or dependencies. If you notice a logical contradiction in the developer's feature file specifications, alert them immediately rather than writing broken code.

feature-branch.mdc

Feature branch gate

Constitution Principle 2: product implementation lives on `feature/[N]-[short-name]` , branched from `dev` . Never implement features on `main` / `master` / `dev` .

When this applies

Before creating or editing **product code** (backend/frontend app code, feature tests that implement a feature) for a numbered feature:

- 1. Identify **Feature N** and the branch from the spec header (`**Branch pattern:** \ feature/N-...```) or catalog.
- 2. Run: `git branch --show-current`
- 3. Gate the result (below).

Does **not** block: editing `features/*.md` , `.cursor/rules/` , ADRs, docs, or Agility/scripts while clarifying specs.

Gate

Current branch	Action
<code>feature/N-...</code> matching the feature being implemented	Proceed
Wrong <code>feature/M-...</code> ($M \neq N$)	Stop. Tell the human which branch is required. Do not switch.
<code>main</code> / <code>master</code>	Refuse product-code edits. Tell the human which branch to create/check out.
<code>dev</code>	Refuse product-code edits. Tell the human which branch to create/check out.

Agent must not

- Run `git checkout` , `git switch` , or `git branch` to create or change branches for feature work.
- Create the feature branch on the user’s behalf.

What to tell the human

State clearly, for example:

Product code for Feature N must be on `feature/N-short-name` (from the feature spec).
You are on `<current-branch>`.
Create or check out that branch from `dev` first, then ask me to continue.

Use the exact short-name from the feature spec (e.g. `feature/3-todo-list-item-management`). Optional helper they can run themselves:

```
git checkout dev  
git checkout -b feature/N-short-name
```

After they switch

Confirm with `git branch --show-current` , then implement only on that branch. Merge back to `dev` when the feature is done (human/PR) — do not merge feature branches into `main` .

quality-attributes.mdc

Quality attributes (NFR literacy)

App-wide bars live in [docs/nfr/quality-attributes.md](#) . Feature specs still authorize **what** to build (constitution Principle 1). This rule teaches **how to read Status** — it does **not** turn every Deferred number into a coding requirement.

Status → agent behavior

Status	Meaning	When generating code
Accepted	In force for this product	Do not regress. Follow linked ADRs/rules/tests (security, ownership, logging, SDD maintainability).
Deferred	Documented example / backlog	Guidance only. Do not add load tests, WCAG tooling, HA, or “meet p95” work unless a feature FR-00N , SC-00N , Gherkin scenario, or the human explicitly asks.
Out of scope	Explicit non-goal	Do not invent multi-region HA, i18n frameworks, horizontal scale, etc.
Accepted (minimal)	Thin bar in force	Meet the stated Approach (e.g. Winston logging); do not expand into a full observability platform.

Do

- Prefer patterns already required by **Accepted** rows and their **Links** (e.g. `security.mdc` , ADR-0002).
- Put feature-local quality in that feature’s **Requirements (FR-00N)** + Gherkin when it must be proven.
- If an app-wide bar’s Target/Approach/Status changes, update `docs/nfr/quality-attributes.md` (and ADR/rule when Approach changes).

Do not

- Treat illustrative Target numbers on **Deferred** / **Out of scope** rows as CI gates or implementation tickets.
- Over-engineer to “hit” Deferred SLOs (caching for latency, fake i18n, cluster deploy).
- Skip feature specs because an NFR row exists — NFRs do not authorize new product behavior alone.

project-structure.mdc

Project Structure

Split frontend and backend in one repository. Specs in `features/` are the source of truth.

Directory Layout

```
todo-speckit/
  features/           # SDD specs
  frontend/          # Vue 3 + Vite + Vuetify 4
    public/          # .htaccess for SPA deploy
    src/
      views/         # Route-level pages
      components/    # Reusable UI (forms, nav, dialogs)
      services/      # Axios API modules (*Services.js)
      config/        # utils.js (localStorage, helpers)
      utils/         # Pure helper functions
      plugins/       # vuetify.js, webfontloader.js
      router.js
      main.js
      App.vue
    vite.config.js
    package.json
  backend/           # Node + Express + Sequelize (ES modules)
    server.js
    app/
      routes/        # Express routers; index.js mounts all
      controllers/   # Request handlers
      models/        # Sequelize models; index.js wires associations
      authorization/  # authenticate, scope helpers, role guards
      config/        # db, auth, logger, sequelizeInstance
      scripts/       # Incremental schema/data scripts
    package.json
```

Naming Conventions

- Frontend services: `camelCase + Services.js` (e.g. `authServices.js` , `todoServices.js`).
- Backend files: `resource.controller.js` , `resource.routes.js` , `resource.model.js` .
- Multi-word URL segments: `kebab-case` (`/todo-lists`).

Environment Variables

Backend (`backend/.env`):

- `DB_HOST` , `DB_USER` , `DB_PW` , `DB_NAME` — MySQL connection (default database: `todospeckit-db`).
- `AUTH_SECRET` — JWT signing secret (required in production).
- `PORT` — default `3200` .
- `NODE_ENV` — `development` | `production` | `test` .

- `SEQUELIZE_SYNC ALTER` — set `true` to run `sync({ alter: true })` in dev only.

Frontend: use `import.meta.env.DEV` for dev/prod API base URL; `cross-env APP_ENV=development` in npm scripts.

Backend test env (`backend/.env.test`): copy from `backend/.env.test.example` ; uses `NODE_ENV=test` and `DB_NAME=todospeckit-db-test` .

Dev Ports

- Frontend Vite: `8082`
- Backend Express: `3200`
- CORS origin must match the frontend dev URL.

Test Commands

- Root: `npm test` (runs backend + frontend)
- Backend: `cd backend && npm test` (Jest + supertest)
- Frontend: `cd frontend && npm test` (Vitest + `@vue/test-utils`)
- Copy `backend/.env.test.example` → `backend/.env.test` before running backend tests locally.

api-conventions.mdc

Backend API Architecture

Stack

- Node.js "type": "module" — import/export only.
- Express 4, mounted at /todo/ .
- Sequelize 6 + mysql2; config in app/config/db.config.js via dotenv.
- Separate sequelizeInstance.js ; models import it and register in models/index.js .
- bcryptjs + jsonwebtoken + Session model for auth.
- winston + morgan (morgan streams to winston).

server.js Responsibilities

- db.sequelize.sync() on startup; optional { alter: true } when SEQUELIZE_SYNC_ALTER=true .
- CORS with credentials: true and frontend origin.
- express.json() and urlencoded({ extended: true }) .
- Mount routes: app.use("/todo", routes) .
- Export app as default; call listen() only when NODE_ENV !== "test" .

Route Layout

- app/routes/index.js — central router registering each *.routes.js .
- Auth at mount root: POST /register , POST /login , POST /logout , POST /reset-password (dev only).
- Resource routes: router.use("/lists", ListRoutes) etc.
- Nested children on parent router: GET /lists/:listId/todos .

Todo REST shape:

- GET/POST /todo/lists
- GET/POST /todo/lists/:listId/todos
- PUT/DELETE /todo/todos/:id

Controller Pattern

```
import db from "../models/index.js";
import logger from "../config/logger.js";

const exports = {};
exports.findAll = async (req, res) => { /* ... */ };
export default exports;
```

- Parse route IDs: parseInt(req.params.id, 10) ; reject NaN .
- Use logger.debug/warn/error — never console.log .
- Sequelize include for associations; order for stable list sorting.

JSON Responses

- **Success:** res.send(data) or res.status(201).send(created) .

- **Failure:** `res.status(4xx).send({ message: "Human-readable explanation." })` .
- **404 format:** `{ message: `List with id=${id} not found.` }`
- Do **not** use `{ success, data }` envelopes.

Model Conventions

- Sensitive fields excluded via `defaultScope` (e.g. User password).
- Use `Model.unscoped()` only when password compare is required at login.
- Associations and `hasMany` / `belongsTo` defined in `models/index.js` , not scattered in controllers.

Schema Changes

Incremental scripts in `app/scripts/` — one concern per file, runnable via `npm run <script>` .

Protected Routes

```
router.get("/", [authenticate], controller.findAll);
```

`authenticate` looks up Bearer token in Session table, checks expiration, sets `req.user` .

auth-patterns.mdc

Auth & Session Patterns

Session Model

- Fields: `token` , `email` , `expirationDate` , `userId` (FK to users).
- Token stored server-side; client sends it as Bearer header on each request.
- Session TTL: 24 hours (`expiresIn: 86400` on JWT; matching `expirationDate` on row).
- On login, reuse non-expired session for same email before issuing a new token.

Login Response Payload

Return flat JSON (not wrapped):

```
{
  "userId": 1,
  "username": "jdoe",
  "email": "jdoe@example.com",
  "fName": "Jane",
  "lName": "Doe",
  "role": "worker",
  "token": "<jwt>"
}
```

Frontend Auth State

- Store full login response in `localStorage` key `user` via `Utils.setStore("user", res.data)` .
- `authServices.js` : `loginUser` , `logoutUser` methods calling `apiClient.post` (see `frontend-services.mdc`).
- Logout clears server session, then `Utils.removeItem("user")` , then route to login.
- `router.beforeEach` : if no `user` in store and route is not login → redirect to login.
- Axios token attachment and 401 redirect: `frontend-services.mdc` .

Registration (when spec requires)

- Hash password server-side before `User.create` .
- Validate required fields in controller; return `400` with `{ message }` .
- Mirror the same field rules in the Vue form with Vuetify validation rules.

frontend-services.mdc

Frontend Services & Routing

File Roles

- `views/` — pages registered in `router.js` .
- `components/` — shared forms, nav, dialogs.
- `services/services.js` — shared axios instance; domain modules in `*Services.js` .
- `config/utils.js` — `setStore` , `getStore` , `removeItem` (localStorage JSON).
- `utils/` — pure helpers only; no HTTP calls.

Domain Service Modules

- One file per domain: `authServices.js` , `todoServices.js` , etc.
- Default-export an object of methods; each method calls `apiClient` and returns the promise.
- No axios calls inside views or components — always go through `*Services.js` .

Axios Client (`services/services.js`)

- Dev `baseUrl` : `http://localhost:3200/todo/` ; prod: `/todo/` .
- `withCredentials: true` .
- `transformRequest` : attach `Authorization: Bearer <token>` from `Utils.getStore("user")` .
- `transformResponse` : on unauthorized message, clear user and `Router.push({ name: "login" })` .
- FormData uploads: delete `Content-Type` header so the browser sets the boundary.

Router Conventions (`router.js`)

- vue-router 4 with `createWebHistory` .
- Named routes (`name: "editTodo"`).
- `props: true` on routes with `:id` params.
- `router.beforeEach` — auth and role guards per `auth-patterns.mdc` .

Cross-Component Events

Dispatch `window CustomEvent` s (e.g. `user-logged-in`) when auth state changes so layout components can refresh without prop drilling.

Production Deploy

Copy `public/.htaccess` to `dist/` on build — SPA fallback rewrite and cache headers for hashed assets.

security.mdc

Security & Access Control

Authentication Flow

1. `POST /login` — validate `username + password`; normalize username: `trim().toLowerCase()` .
2. Look up user; compare password with `bcrypt.compare()` against hash from `User.unscoped()` .
3. Reuse valid Session row if token not expired; otherwise create new JWT + Session row.
4. Return user payload including `token` , `userId` , `role` (never return password hash).
5. `POST /logout` — clear token on Session row.
6. `AUTH_SECRET` from environment; no hardcoded secrets in production.

Request Authentication

- `authenticate` middleware reads `Authorization: Bearer <token>` .
- Validates token exists in Session table and `expirationDate >= Date.now()` .
- Sets `req.user = { id, role, organizationId }` from joined User row.
- Missing/invalid → `401` with `{ message: "Unauthorized! ..." }` .

User-Scoped Data (Todo App)

- Every list/todo query must include `userId: req.user.id` in the `where` clause.
- On create, set `userId` from `req.user.id` — never from request body.
- Before update/delete, load the row and verify ownership; return `404` if not found or not owned (do not leak existence via `403`).

Row-Level Access Helpers

Centralize repeated checks in `app/authorization/` :

```
export const getAccessibleListOrNull = async (req, listId) => {
  const row = await List.findOne({ where: { id: listId, userId: req.user.id } });
  return row ?? null;
};
```

Controllers call helpers; do not duplicate scope logic inline.

Role Guards

Export named middleware from `authorization/authorization.js` :

- `requireAdmin` — role is `admin` or `superadmin` .
- `requireSuperAdmin` — role is `superadmin` only.

Password Rules

- Hash with `bcryptjs`, `SALT_ROUNDS = 10` .
- Minimum length enforced at controller (e.g. 8 chars on reset).
- `reset-password` disabled when `NODE_ENV === "production"` .

Optional Multi-Org Scoping (future)

If org tenancy is added, centralize filters in `authorization/tenantScope.js` :

- `sessionTenantScopeWhere(req)` for standard users.
- `X-Acting-Organization-Id` header for superadmin org switching.
- Never trust client-supplied `organizationId` on write operations.

testing-standards.mdc

Quality Assurance & Test Verification

Every feature in `features/` must name its test coverage. Ghost tests are forbidden.

See `features/framework.md` → **Test traceability** for the full convention.

Traceability structure

1. **File header** — Feature N title + Spec: `features/feature-N-...md`
2. `describe("Feature N - ...")` — outer wrapper
3. `describe("US-N.n - Story title")` — matches AC `###` heading in spec
4. `it("Scenario title")` — exact Gherkin `#### Scenario: name from spec`

Harness smoke tests (`backend/tests/app.test.js` , `frontend/tests/App.test.js`) are exempt from story nesting.

Backend (Jest + supertest)

- Import the Express `app` export (server does not listen in `NODE_ENV=test`).
- Integration-test each endpoint with supertest.
- Per success case, also test:
 1. Missing/invalid input → `400` with `{ message }` .
 2. Missing/expired token → `401` .
 3. Another user's resource → `404` (scoped access).
- Use a dedicated test database (`todospeckit-db-test`); copy `backend/.env.test.example` to `backend/.env.test` .

Frontend (Vitest + @vue/test-utils)

- Mount views/components with reactive state.
- Assert client-side validation blocks submit before API call.
- Assert loading and error UI states on promise resolve/reject.

SDD link

Each Gherkin scenario in `features/feature-*.md` maps to at least one `it` block. Traceability: **US-N.n** → scenario → test; functional rules live in **FR-00N**; measurable bars in **SC-00N**. No `expect(true).toBe(true)` .

ui-style-system.mdc

UI Style System

OC Academic Edition — a hybrid design system merging modern SaaS functionality with traditional academic visual heritage.

Vuetify 4 only — no Tailwind, no Options API.

Stack

- Vue 3 `<script setup>` exclusively.
- Vuetify 4 via `vite-plugin-vuetify ( autoImport: false )`.
- `@mdi/font` icons; theme defined in `plugins/vuetify.js`.
- Font: **Inter** (`Inter, sans-serif`) — load via web fonts; do not fall back to system UI stacks as the primary face.

Brand & Color Tokens

Hex values live **only** in `plugins/vuetify.js`. Views and components use Vuetify color names / CSS variables — never raw hex.

Token	Hex	Role
primary	#801328	Signature Deep Maroon — CTAs, headings, key accents
primary-container	#FFDAD9	Soft maroon tint surfaces (chips, selected backgrounds)
on-primary	#FFFFFF	Text/icons on primary
secondary	#775656	Supporting actions, muted chrome
secondary-container	#FFDAD9	Soft secondary tint surfaces
on-secondary	#FFFFFF	Text/icons on secondary
surface	#F9F9FF	Cards, panels, elevated surfaces
surface-variant	#E7E0E1	Subtle fills, dividers adjacent
background	#FFFFFF	App page background
outline	#857373	Borders, hairlines, inactive strokes

Map these into Vuetify's `theme.themes.light.colors` (and Material-style aliases such as `on-primary` where supported).

Typography

Token	Value
Font family	<code>Inter, sans-serif</code>
Base size	<code>16px</code>
Scale	<code>1.25 (Major Third)</code>

Headings: weight `700` , color `primary` (`#801328`).

Body: weight `400` , color `#1C1B1F` .

Suggested scale from base $\times 1.25$: $16 \rightarrow 20 \rightarrow 25 \rightarrow 31.25 \rightarrow 39$. Prefer Vuetify type props / theme typography over ad-hoc `font-size` in components.

Spacing

- Base unit: `4px`
- Scale (px): `0`, `4`, `8`, `16`, `24`, `32`, `48`, `64`
- Prefer Vuetify spacing utilities (`pa-4` , `ma-2` , gap props) that align to this scale — avoid arbitrary pixel values.

Shape & Elevation

- **Border radius:** `8px` ("Round Eight") — set as the default in the Vuetify theme / defaults; use consistently on cards, dialogs, inputs, and buttons.
- **Elevation — low:** `0 1px 3px rgba(0,0,0,0.12)`, `0 1px 2px rgba(0,0,0,0.24)`
- **Elevation — medium:** `0 4px 6px rgba(0,0,0,0.1)`, `0 1px 3px rgba(0,0,0,0.08)`

Map low $\rightarrow$ light chrome (list rows, subtle panels); medium $\rightarrow$ cards and primary dialogs. Prefer theme defaults / `elevation` props over inline `box-shadow` unless extending the theme.

App Shell (`App.vue`)

```
<v-app>
  <MenuBar />
  <v-main><router-view /></v-main>
</v-app>
```

Hide `MenuBar` on login and register routes (Feature 1). Introduce `MenuBar` in Feature 2.

Form Components

- Reusable forms use `v-model` via `defineProps({ modelValue }) + emit("update:modelValue")` .
- Vuetify inline rules: `[(v) => !!v?.trim() || "Required"]` .
- Add/Edit/View modes via props (`readOnly` , `isAddMode`) — one form component, multiple views.

Vuetify Defaults

- Layout: `<v-container>` , `<v-row>` , `<v-col cols="12" md="6">` .
- Inputs: `<v-text-field density="comfortable" rounded="lg">` (8px family).
- Buttons — primary actions: `<v-btn color="primary" variant="elevated" class="oc-cta" :loading="loading">` .
- Buttons — secondary / cancel: `<v-btn color="secondary" variant="text">` or `variant="outlined"` .
- Buttons — **label size**: all primary labeled actions (+ New List , Add , Edit Profile , dialog confirms) use class `oc-cta` (0.875rem / medium weight — matches Edit Profile). Defined in `App.vue` . Do **not** nest these buttons inside `<v-card-title>` — title typography inherits and makes the label look oversized. Prefer `<v-card-item>` with `#append` for title + CTA layouts. Reserve `size="small"` for icon-only row actions (rename / delete).
- Cards: `<v-card rounded="lg">` on surface ; prefer medium elevation for content cards.
- Errors: `<v-alert type="error" density="compact">` .

- Dialogs: `<v-dialog>` for create / rename / confirm flows; round-eight radius; primary confirm + secondary/text cancel.
- Theme color tokens only in components — hex values belong in `plugins/vuetify.js` .

View States

Every data-driven view handles: **loading** (`:loading` or spinner), **empty** (zero-length message), **error** (ref + alert).

agent-behavior.mdc

Agent behavior

Reduce common LLM coding mistakes. Biases **caution over speed**; for trivial one-line fixes, use judgment.

These guidelines **complement** the [constitution](#) and stack rules. They do **not** authorize product scope — that stays in `features/feature-*.md`.

1. Think before coding

Don't assume. Don't hide confusion. Surface tradeoffs.

- State assumptions explicitly. If uncertain, ask.
- If multiple interpretations exist, present them — don't pick silently.
- If a simpler approach exists, say so. Push back when warranted.
- Unclear or contradictory **feature spec** → stop, name the conflict (constitution Principle 6). Do not invent FR/Gherkin.

2. Simplicity first

Minimum code that solves the authorized problem. Nothing speculative.

- Implement only what the active feature's **FR-00N**, Screen/API/Data sections, and Gherkin require — no extras.
- No abstractions for single-use code; no unrequested "flexibility" or config.
- No error handling for impossible scenarios.
- If you wrote far more than needed, rewrite smaller.

Ask: would a senior engineer call this overcomplicated? If yes, simplify.

3. Surgical changes

Touch only what you must. Clean up only your own mess.

- Don't "improve" adjacent code, comments, or formatting.
- Don't refactor unrelated working code.
- Match existing style (constitution Principle 5 / stack `.mdc` rules).
- Unrelated dead code → mention it; don't delete unless asked.
- Remove imports/vars/functions **your** change made unused; don't sweep pre-existing dead code.

Every changed line should trace to the user request **and** the authorizing spec (when implementing a feature).

4. Goal-driven execution

Define success criteria. Loop until verified.

Map work to checks you can run:

Ask	Success looks like
Implement Feature N	Test Coverage Map its exist and npm test passes; reference updated if API/schema/rules changed
Add validation	Tests for invalid inputs pass

Fix a bug	Reproducing test fails, then passes
Refactor	Tests pass before and after

For multi-step work, a brief plan with verify steps:

```
1. [Step] → verify: [check]
2. [Step] → verify: [check]
```

Prefer strong criteria (mapped scenarios, commands) over "make it work."

Working if: smaller diffs, fewer speculative abstractions, clarifying questions before wrong implementation.

Part 2: Architecture Decision Records

README.md

Architecture Decision Records (ADRs)

Durable **why** decisions for cross-cutting concerns that outlive any single feature spec.

Artifact	Question
Constitution	What are the non-negotiable laws?
Cursor rules	How must we implement (patterns)?
ADRs (docs/adr/)	Why did we choose this approach?
Feature specs	What must the product do?
Quality attributes	What app-wide NFR / ility bars apply?
Reference	What exists on dev now?

ADRs do **not** replace feature specs or Cursor rules. They capture context, alternatives, and consequences when a choice affects multiple features or the whole stack.

Student guide: [writing-adrs.md](#) (PDF) — when to write an ADR, naming, section principles, and checklist.

When to write an ADR

Situation	Write an ADR?
New product behavior for one feature	No — use features/feature-N-*.md
Ongoing coding pattern for the team	No — use .cursor/rules/*.mdc
Significant stack or architecture choice	Yes
Security or data-isolation model	Yes
Deviation from an existing rule	Yes — then update the rule

Write the ADR **before** or **with** the first feature that depends on the decision. Link it from affected feature specs (****Related:**** in the header).

File naming

```
docs/adr/  
  README.md           ← this file  
  writing-adrs.md      ← student guide (when / how to write)  
  NNNN-short-kebab-title.md ← one decision per file
```

- **Number:** four digits, sequential (`0001` , `0002` , ...). Never reuse a retired number.
- **Title:** short, specific, kebab-case.
- **Status:** `Proposed` | `Accepted` | `Deprecated` | `Superseded by ADR-NNNN`

Template

Copy the block below into a new file and fill in each section.

```
# ADR-NNNN: Short title
```

```
**Status:** Proposed | Accepted | Deprecated | Superseded by [ADR-XXXX](XXXX-title.md)
```

```
**Date:** YYYY-MM-DD
```

```
**Deciders:** team / role names
```

Context

What problem or constraint forced a decision? What was unclear?

Decision

What we chose, in one or two sentences. Be specific (technologies, boundaries, invariants).

Consequences

Positive

— ...

Negative / tradeoffs

— ...

Alternatives considered

Option	Why not
-----	-----
... ...	

Related artifacts

- Feature specs: ...
- Cursor rules: ...
- Supersedes / superseded by: ...

Index

ADR	Title	Status
0001	Client-server architecture for multi-user todo data	Accepted
0002	Layered security architecture	Accepted
0003	MySQL relational database	Accepted

0001-client-server-multi-user-architecture.md

ADR-0001: Client–server architecture for multi-user todo data

Status: Accepted

Date: 2026-07-07

Deciders: OC CS Speckit project (SDD kit; Todo example application)

Context

The **Todo** example application shipped with OC CS Speckit is a **multi-user** todo app: each registered user owns private lists and items. No user may read or modify another user's data. The kit must be teachable as a Spec-Driven Development reference — clear boundaries between specification, frontend, backend, and tests.

We needed to decide:

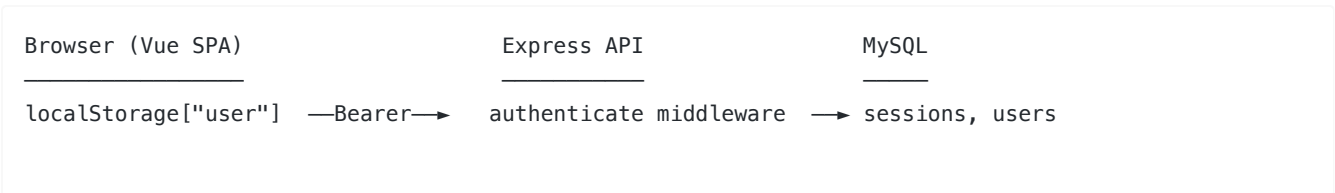
1. Whether the browser holds authoritative state or only talks to a shared server.
2. How to identify the caller on every API request.
3. How to enforce per-user data isolation consistently across features.

A single-user or offline-first design (localStorage as source of truth, optional sync) would simplify the SPA but would not model real multi-tenant boundaries or shared MySQL persistence.

Decision

Adopt a **classic client–server split** with a **stateless REST API** and **server-enforced user scoping**:

Layer	Choice
Client	Vue 3 SPA (Vite), Vuetify 4, axios
Server	Node.js + Express + Sequelize (ES modules)
Database	MySQL — single shared database, rows scoped by <code>userId</code>
Transport	JSON over HTTPS; API base path <code>/todo/</code>
Auth	Username + password; bcrypt hashes; JWT + Session table (token stored server-side, revocable on logout)
Client session hint	Login response stored in <code>localStorage</code> key <code>user</code> ; axios attaches <code>Authorization: Bearer <token></code> on every request
Authorization	<code>authenticate</code> middleware sets <code>req.user.id</code> ; all <code>list/todo</code> queries filter by <code>userId</code> ; <code>create</code> writes use <code>req.user.id</code> , never <code>body</code> ; cross-user access returns 404 (not 403)
Repo layout	Monorepo: <code>frontend/</code> + <code>backend/</code> + <code>features/</code> <code>specs</code>



router guards (UI)

controllers + auth helpers lists, todos
userId in every WHERE clause

Invariants (must hold in every feature):

1. The server is the **source of truth** for lists, todos, and profile data.
2. Every authenticated request resolves to **exactly one** `req.user.id` from a valid session row.
3. **No endpoint** returns or mutates rows owned by another user.
4. The client never sends a trusted `userId` on create — the server assigns ownership.

Consequences

Positive

- Clear SDD layers: feature specs → API contract → implementation → Jest/Vitest proof.
- Realistic multi-user security model suitable for classroom and production-style review.
- Session revocation on logout (empty token on Session row) — not possible with JWT-only, client-only auth.
- Same patterns extend to Features 2–5 without re-deciding architecture.

Negative / tradeoffs

- Requires a running MySQL instance and backend for full-stack work (not a static or offline demo).
- `localStorage` session is a **convenience cache** for UX (router guards, display name); it is not authoritative — 401 clears it and redirects to login.
- Per-user row scoping in SQL is simpler than org/team tenancy; multi-org would need a new ADR and schema work.
- CORS and two dev ports (`8082` frontend, `3200` backend) add local setup steps.

Alternatives considered

Option	Why not
localStorage-only todos (no backend)	No shared database, no real multi-user isolation, does not match course API/testing goals.
JWT in cookie only, no Session table	Harder to revoke on logout; server cannot invalidate a stolen token without extra infrastructure.
GraphQL or tRPC	Heavier stack; REST + flat JSON matches existing rules and Agility export simplicity.
403 Forbidden on cross-user IDs	Leaks that a resource exists; 404 treats other users' rows as not found (see <code>security.mdc</code>).
Server-rendered Vue (SSR)	Out of scope for Vite SPA starter; auth still needs the same session model.

Related artifacts

- ADRs: [ADR-0002 — Layered security architecture](#), [ADR-0003 — MySQL relational database](#)
- C4 diagrams: [docs/arch_diagrams/](#) (context, container, component)
- Feature specs: [Feature 1 — User Authentication](#) (identity foundation); Features 2–3 (list/todo isolation)
- Cursor rules: [auth-patterns.mdc](#), [security.mdc](#), [frontend-services.mdc](#), [project-structure.mdc](#)
- Reference: [api.md](#), [data-model.md](#)

0002-security-architecture.md

ADR-0002: Layered security architecture

Status: Accepted

Date: 2026-07-07

Deciders: OC CS Speckit project (SDD kit; Todo example application)

Context

[ADR-0001](#) establishes the client-server split and per-user data boundaries. Security still needed an explicit model for **where** trust is enforced, **how** sessions are validated, and **what** the application deliberately does not implement in v1.

Threats relevant to this app:

- User A accessing user B's lists, todos, or profile.
- Stolen or replayed session tokens after logout or expiry.
- Client tampering (`userId` in request body, ID enumeration).
- Credential disclosure (password hashes in API responses, weak storage).
- Relying on frontend-only checks (router guards, form validation) as security controls.

The security model must be teachable, testable via Gherkin scenarios, and consistent across Features 1–5 without re-deciding per endpoint.

Decision

Adopt a **layered security architecture** with the **API as the sole enforcement point** and **defense in depth** on the client for UX only.

Trust boundaries



- sessions table for revocable tokens

Layer 1 — Authentication (who is calling?)

Control	Implementation
Credentials	Username + password; username normalized <code>trim().toLowerCase()</code>
Password storage	<code>bcryptjs</code> , <code>SALT_ROUNDS = 10</code> ; minimum 8 characters at registration/update
Session token	JWT signed with <code>AUTH_SECRET</code> ; also stored in <code>sessions</code> table with <code>expirationDate</code>
Token lifetime	24 hours
Request proof	Authorization: Bearer <token> on every protected route
Validation	<code>authenticate</code> middleware: token must exist in DB, not expired, joined to a user
Logout	Clear token on session row (server-side revocation)
Identity on request	<code>req.user = { id, role }</code> — controllers use <code>req.user.id</code> only

Hybrid JWT + Session table: the JWT carries the token value; the database row is the revocation and expiry gate. Logout and expiry are enforceable without a token blacklist service.

Layer 2 — Authorization (what may this user do?)

Control	Implementation
Row-level scope	Every list/todo query includes <code>userId: req.user.id</code> in the <code>WHERE</code> clause
Create ownership	Set <code>userId</code> from <code>req.user.id</code> ; ignore client-supplied <code>userId</code> in body
Update/delete	Load via <code>getAccessibleListOrNull</code> , <code>getAccessibleTodoOrNull</code> , or <code>getAccessibleUserOrNull</code>
Cross-user access	Return 404 with a not-found message — never 403 (avoids confirming resource existence)
Profile access	User may only GET/PUT their own <code>userId</code> ; <code>:id</code> must match <code>req.user.id</code>
Centralization	All scope checks live in <code>backend/app/authorization/</code> — controllers do not inline duplicate logic

Layer 3 — Client hardening (UX, not security)

Control	Purpose
<code>router.beforeEach</code>	Redirect unauthenticated users away from protected routes
<code>axios transformRequest</code>	Attach Bearer token from <code>localStorage</code>
<code>axios transformResponse</code>	On 401, clear user and redirect to login
Vuetify form rules	Block invalid submits before network (email format, password length, required fields)

Rule: passing a Vitest router test or skipping client validation must **not** grant access; Jest + supertest against the API is the security proof.

Layer 4 — Secrets and configuration

Secret / config	Location
AUTH_SECRET	backend/.env — never committed
DB credentials	backend/.env / backend/.env.test
Password reset endpoint	Disabled when NODE_ENV === "production"

Roles (foundation only)

New users receive role `worker`. Middleware hooks (`requireAdmin` , `requireSuperAdmin`) exist for future admin features but are not used by current todo CRUD. Todo data is scoped by **user**, not role.

Explicitly out of scope (v1)

Documented deferrals — not security holes by omission in the teaching model, but not implemented:

Control	Status
Rate limiting / brute-force lockout	Out of scope
CSRF tokens	Not required for Bearer-header SPA API (no cookie session)
OAuth / social login	Out of scope (Feature 1)
Email verification / password reset flow	Out of scope
Field-level encryption at rest	Out of scope
Multi-org / tenant isolation	Future ADR if added
Content Security Policy / Helmet hardening	Recommended for production deploy; not spec-gated

Consequences

Positive

- Single enforcement point (`authenticate` + authorization helpers) — easy to audit and test.
- Gherkin scenarios in Features 1–4 map directly to security behavior (401, 404, ownership).
- Revocable sessions without Redis or a dedicated token blacklist.
- 404-on-cross-user pattern reduces information leakage in a shared-database multi-user app.
- Centralized helpers prevent scope-check drift across controllers.

Negative / tradeoffs

- Bearer token in `localStorage` is vulnerable to XSS — mitigated by framework defaults and no `v-html` on user content, but not as strong as HttpOnly cookies.
- No rate limiting leaves login open to online brute force in a public deployment.
- Per-user scoping does not scale to shared lists or teams without schema and ADR changes.
- Client and server both validate inputs — duplicate logic maintained deliberately (UX vs enforcement).

Alternatives considered

Option	Why not
--------	---------

Frontend-only auth (router guard sufficient)	Trivially bypassed via curl; fails constitution Principle 3.
403 on cross-user ID	Confirms resource exists to an attacker probing IDs.
Trust userId from request body on create	Client can assign rows to other users.
JWT only (no Session table)	Cannot revoke on logout without extra infrastructure.
Cookie-based session without Bearer	Complicates SPA CORS/dev setup; CSRF becomes mandatory.
Inline scope checks per controller	Duplication risk; already rejected in favor of <code>getAccessibleOrNull</code> helpers.
RBAC for todo ownership	Overkill; worker role + userId FK is sufficient for private todos.

Related artifacts

- ADRs: [ADR-0001 — Client-server multi-user architecture](#)
- Feature specs: [Feature 1](#) (auth); [Features 2–3](#) (list/todo isolation); [Feature 4](#) (profile scope)
- Cursor rules: [security.mdc](#), [auth-patterns.mdc](#), [frontend-services.mdc](#)
- Implementation: `backend/app/authorization/authorization.js`
- Tests: `backend/tests/authenticate.test.js` , `backend/tests/auth.test.js` , ownership scenarios in `lists.test.js` , `todos.test.js` , `users.test.js`

0003-mysql-relational-database.md

ADR-0003: MySQL relational database

Status: Accepted

Date: 2026-07-07

Deciders: OC CS Speckit project (SDD kit; Todo example application)

Context

The **Todo** example application persists multi-user identity, sessions, lists, and todos. The data is inherently **relational**: users own lists; lists contain todos; sessions belong to users. [ADR-0001](#) requires a shared server-side database; [ADR-0002](#) requires row-level ownership enforced in every query.

We needed to decide:

1. **Relational SQL vs document/NoSQL** for this domain.
2. **Which SQL engine** fits a classroom + XAMPP-style local setup.
3. **How** the Node backend talks to the database (ORM, schema evolution, tests).

The stack must work on developer laptops (often XAMPP with MySQL already installed), support foreign keys and transactions, and stay simple enough to teach alongside Sequelize models and Jest integration tests.

Decision

Use **MySQL** as the production database with **Sequelize 6** as the ORM and **relational, normalized tables** scoped by `userId` foreign keys.

Stack

Layer	Choice
Database	MySQL 5.7+ / 8.x (via XAMPP, Docker, or native install)
Driver	mysql2
ORM	Sequelize 6 (ES modules)
Config	backend/app/config/db.config.js + sequelizeInstance.js; credentials from .env
Default database	todospeckit-db
Test database	Separate todospeckit-db-test (backend/.env.test)

Schema model

Four core tables with explicit foreign keys (see [data-model.md](#)):

```
users — sessions
      |
      |— lists — todos
      |
      |— todos (direct userId for authorization queries)
```

Table	Purpose
users	Accounts; bcrypt password hash; unique email and username
sessions	Revocable Bearer tokens; expirationDate; FK → users.id
lists	Per-user todo lists; FK → users.id
todos	Items in a list; FK → lists.id + users.id; onDelete: CASCADE from list

Design rules:

- **Normalized relational schema** — no embedded todo arrays in list documents.
- **userId on lists and todos** — enables authorization WHERE clauses without joins-only assumptions.
- **Cascade delete** — removing a list deletes its todos (US-3.6).
- **DATEONLY for dueDate** — date-only semantics without timezone complexity (Feature 5).
- **Timestamps** — Sequelize createdAt / updatedAt on all tables.
- **Uniqueness** — email and username enforced at DB + controller.

Schema evolution

Environment	Strategy
Development	sequelize.sync({ alter: true }) on server start when SEQUELIZE_SYNC_ALTER=true (default in .env.example)
Production	sync() without alter; schema changes require explicit migration discipline (out of scope for v1 teaching model)
Tests	sync({ force: true }) in Jest beforeAll — drops and recreates tables per suite; resetTestDatabase() truncates between tests

No checked-in Sequelize migration files in v1 — schema is defined in `backend/app/models/*.model.js` and synced. Feature specs authorize schema changes; `features/reference/data-model.md` is updated in the same feature PR when schema changes (see [Agent implementation request](#)).

Connection handling

- Connection pool: `max: 5`, `min: 0` in `db.config.js`.
- Server refuses to start if sync fails outside `NODE_ENV=test`.
- Tests close `db.sequelize` in `afterAll` to avoid connection leaks across suites.

Query patterns

- Sequelize model definitions + `findOne` / `findAll` with explicit `where` clauses.
- Authorization helpers add `userId: req.user.id` to every scoped lookup.
- `User.unscoped()` only when bcrypt password comparison requires the hash column.

Consequences

Positive

- Natural fit for user → list → todo hierarchy and FK integrity.
- MySQL ships with XAMPP — low friction for local full-stack development.
- Sequelize models map cleanly to SDD **Data Model Requirements** sections in feature specs.
- Separate test database prevents dev data loss during `force: true` test sync.
- `alter: true` in dev speeds iteration without hand-written migrations for coursework.

- SQL `WHERE userId = ?` aligns with [ADR-0002](#) authorization model.

Negative / tradeoffs

- MySQL must be installed and running — not zero-dependency like SQLite file DB.
- `sync({ alter: true })` is unsafe for production schema changes; real deployments need migrations (deferred).
- Sequelize adds abstraction weight vs raw SQL.
- No read replicas, sharding, or connection pooling beyond defaults — single-instance assumption.
- `DATEONLY` avoids timezones but does not support time-of-day due dates (Feature 5 out of scope).

Alternatives considered

Option	Why not
SQLite (file DB)	Simpler setup but weaker classroom alignment with deployed MySQL; concurrent test + dev access is awkward.
PostgreSQL	Excellent choice for production; less universal in XAMPP/LAMP developer environments for this course.
MongoDB / document store	Todo-in-list fits poorly without duplicating ownership; cross-user isolation harder to reason about in specs.
JSON files / in-memory store	No real multi-user persistence; fails ADR-0001.
Prisma	Viable ORM; Sequelize already wired in rules, models, and course materials.
Raw SQL only (no ORM)	More boilerplate; Sequelize matches constitution stack consistency.
Single shared DB for dev and test	Risk of wiping developer data when tests run <code>force: true</code> .

Related artifacts

- ADRs: [ADR-0001](#), [ADR-0002](#)
- Reference: [data-model.md](#)
- Cursor rules: [api-conventions.mdc](#), [project-structure.mdc](#)
- Config: `backend/app/config/db.config.js` , `backend/.env.example` , `backend/.env.test.example`
- Models: `backend/app/models/`
- Tests: `backend/tests/helpers.js` (`syncTestDatabase` , `resetTestDatabase`)

Part 3: Quality Attributes (NFRs)

README.md

Non-Functional Requirements (quality attributes)

Living snapshot of **system characteristics** ("ilities") for this product — performance, reliability, availability, security posture, accessibility, and related bars.

Artifact	Question
Feature specs	What must the product <i>do</i> ?
This folder (<code>docs/nfr/</code>)	What quality bars apply <i>across</i> the product?
ADRs	<i>Why</i> did we choose a particular approach for a quality?
Cursor rules	<i>How</i> must code meet ongoing constraints?
Reference	What API/schema exists on dev now?

NFRs here do **not** authorize new product behavior by themselves. Feature specs still authorize *what* to build. Use this doc to record targets, deferrals, and links to ADRs/rules.

Agent literacy: [.cursor/rules/quality-attributes.mdc](#) (`alwaysApply`) — honor **Accepted**; treat **Deferred** as guidance only; do not invent for **Out of scope**.

Files

File	Contents
quality-attributes.md	Attribute table + Status / Links documentation
writing-quality-attributes.md	Student guide — when/how to write NFR rows (PDF)

Student guide: [writing-quality-attributes.md](#) ([PDF](#))

When to update

When	Action
New app-wide quality bar (or explicit "out of scope")	Update quality-attributes.md
Choose <i>how</i> to meet a bar (caching, HA, auth model, ...)	Write or update an ADR ; link from the attribute row
Ongoing coding constraint for agents	Add/update a <code>.cursor/rules/*.mdc</code> entry; link from the row

Feature-local quality only	Prefer Requirements (FR-00N) / Success Criteria (SC-00N) + Gherkin in that feature; optionally link here
----------------------------	--

Related

- [ADR index](#)
- [SDD framework](#) — feature spec template (**FR-00N**, **SC-00N**, Gherkin)
- [Quality attributes rule](#)
- [Security rule](#)

quality-attributes.md

Quality attributes

App-wide non-functional targets for OC CS Speckit (illustrated by the Todo example application).

Teaching policy: Specs say *what* to build. This table says *how good* the system should be. Only **Accepted** rows (and feature **Requirements (FR-00N)** / **Success Criteria (SC-00N)**) constrain implementation. **Deferred** is the quality backlog / classroom example. **Out of scope** is what not to build. Cursor agents follow [.cursor/rules/quality-attributes.mdc](#) for this literacy — they must **not** treat every Deferred number as always-on.

Column meanings

Column	Meaning
Attribute	Quality characteristic ("ility")
Target	Measure or bar — prefer a number (latency, %, count, level). Figures below are illustrative classroom examples , not production SLOs unless Status is Accepted and tests enforce them.
Approach	How the target is realized or limited (stack choice, pattern, explicit non-goal)
How we verify	Tests, manual checks, or N/A
Status	Whether the bar constrains work today (see below)
Links	Where Approach / enforcement lives (see below)

Status values

Status	Meaning	Developer / agent expectation
Accepted	In force for this product	Must not regress; covered by linked rules, ADRs, and/or tests
Accepted (minimal)	Thin bar in force	Meet the stated Approach only; do not expand scope
Deferred	Documented, not enforced yet	Example Target for learning; implement only if a feature spec or instructor requires it
Out of scope	Explicit non-goal for this Todo example / kit demo	Do not design or generate for this bar

Links column

Use **Links** to point at the durable artifacts that explain or enforce the row:

Link type	When to use
-----------	-------------

ADR (docs/adr/...)	Why this Approach was chosen
Cursor rule (.cursor/rules/...)	How agents/code must behave day to day
Code path (e.g. logger.js)	Concrete implementation of a minimal bar
Feature / framework	Process, FR-00N , SC-00N , or Screen Requirements that carry usability/maintainability
—	No separate artifact yet (common for Deferred rows)

When Approach changes, update or add an ADR and refresh **Links**. When agents need a lasting coding constraint, add/update a Cursor rule and link it here.

Update this table when the bar changes. Feature-local bars stay in that feature's **Requirements (FR-00N)** and/or **Success Criteria (SC-00N)** (+ Gherkin when testable).

Attribute	Target	Approach	How we verify	Status	Links
Security	100% of protected routes require auth; 0 cross-user reads/writes in automated tests; other users' resources → 404 (not 403)	Layered API enforcement; ownership isolation	Gherkin + Jest (supertest); Vitest for UX-only guards	Accepted	ADR-0002 , security.md auth-patterns.mdc
Data integrity	100% of list/todo rows have a valid owning userId; 0 orphan associations after CRUD tests	Relational MySQL; foreign keys / Sequelize associations	Jest + schema in data-model	Accepted	ADR-0003
Reliability	Happy-path write success ≥ 99% in local test runs; failed writes return HTTP 4xx/5xx with a body (never empty 200)	Single-process Express; no HA/retry layer	Jest on create/update/delete paths	Deferred	—

Availability	Local demo uptime goal $\geq$ 95% of lab session time; no multi-region SLA	Single-node deploy (XAMPP or similar); no HA	N/A	Out of scope	ADR-0001
Performance	p95 API latency $<$ 200 ms (local XAMPP); dashboard first paint $<$ 2 s on a typical developer laptop	No formal load-test gate in CI yet	Manual / npm run dev (future: timed Jest or k6)	Deferred	—
Scalability	Correct for $\leq$ 30 concurrent classroom users; $\leq$ 500 todos per user without pagination redesign	Multi-user correctness, not horizontal scale	Ownership tests; manual multi-browser check	Out of scope	ADR-0001
Observability	100% of unhandled server errors logged at error; HTTP access logged; retain rotating logs $\geq$ 7 days	Winston console + daily rotate under backend/logs/	Logs present in local runs	Accepted (minimal)	backend/app/config/logg
Usability	New user completes register $\rightarrow$ create list $\rightarrow$ add todo in $\leq$ 3 minutes without help; primary CTAs use labels from Screen Requirements (100% match); $\leq$ 2	Vuetify + Screen Requirements; oc-cta for primary actions; empty states documented per feature	Manual walkthrough; Vitest for labeled CTAs / flows	Deferred	ui-style-system.mdc , feature Screen Requirements

	clicks from dashboard to add a todo on an existing list				
Accessibility (a11y)	Primary flows keyboard-reachable; aim WCAG 2.2 AA for auth + dashboard when audited; 0 unlabeled icon-only CTAs on primary actions	Prefer Vuetify semantic components	Manual / future Vitest a11y	Deferred	ui-style-system.mdc
Internationalization (i18n)	1 locale (en-US); 0 translated string catalogs	No i18n framework	N/A	Out of scope	—
Maintainability	100% of Gherkin scenarios mapped in Test Coverage Map before merge; npm test green; feature PRs typically ≤ 15 files of product code (guideline)	Cursor rules + feature specs as source of truth	Merge checklist; npm test	Accepted	framework.md , constitution.mdc , quality attributes.mdc

Feature-local NFRs

If only one feature needs a bar (e.g. a specific validation or UI responsiveness note):

1. Put it under that feature's **Requirements (FR-00N)** (behavior/rules) or **Success Criteria (SC-00N)** (measurable outcome).
2. Add Gherkin when it must be proven by tests.
3. Optionally add a one-line pointer here ("see Feature N **FR-00N** / **SC-00N**").

Do **not** invent a new feature file solely to "add performance." See [feature spec template](#).

Changing a bar

1. Edit this table (**Target**, **Approach**, **Status**, **Links**). Prefer a number in **Target**.
2. If the **Approach** changes → ADR; put the ADR in **Links**.
3. If agents need a lasting coding constraint → Cursor rule; put the rule in **Links**.
4. If API/schema changed as a result → features/reference/ in the same feature PR (see [Agent implementation request](#)).
5. Promoting **Deferred** → **Accepted** requires verification (tests or documented manual gate) that matches **How we verify**.

writing-quality-attributes.md

Writing Quality Attributes (NFRs)

A student guide for recording **app-wide quality bars** ("ilities") in `docs/nfr/quality-attributes.md` : what they are, how to fill each column, how to set **Status**, and when to use a feature FR/SC instead.

Living table: [quality-attributes.md](#)

Folder index: [README.md](#)

Agent literacy: [.cursor/rules/quality-attributes.mdc](#)

Related: [writing ADRs](#) · [writing feature requirements](#) · [writing feature design](#)

What quality attributes are

Quality attributes (non-functional requirements, or NFRs) describe **how good** the system should be — security posture, performance, observability, maintainability — not **what feature** to build next.

Artifact	Question
Feature specs	<i>What</i> must the product do?
Quality attributes (<code>docs/nfr/</code>)	<i>How good</i> must it be across the product?
ADRs	<i>Why</i> this Approach for a quality?
Cursor rules	<i>How</i> must code meet an Accepted bar day to day?

NFRs here do **not** authorize new product behavior by themselves. A **Deferred** performance number does not mean "add caching this sprint" unless a feature **FR-00N** / **SC-00N**, Gherkin, or an instructor explicitly requires it.

When to edit the NFR table (vs a feature or ADR)

Situation	Write / update...
App-wide bar or explicit non-goal	quality-attributes.md row
<i>How</i> to meet a bar (auth model, DB, logging approach)	ADR — then link it from the row
Ongoing coding constraint for agents	Cursor rule — then link it from the row
Quality that only one feature needs	Feature FR-00N / SC-00N (+ Gherkin); optional one-line pointer in the table
New user-facing capability	Feature spec — not a new NFR row alone

Rule of thumb: if every feature must respect the bar (or we must explicitly refuse to build for it), it belongs in the quality-attributes table.

Examples from this project

Attribute	Status	Why it belongs in NFRs
Security	Accepted	Cross-cutting: auth on protected routes, 0 cross-user leaks, 404 not 403
Data integrity	Accepted	Every list/todo row has owning userId — app-wide invariant
Observability	Accepted (minimal)	Thin Winston logging bar — meet Approach, don't invent a full APM platform
Performance	Deferred	Illustrative p95 / first-paint numbers — not a CI gate yet
Availability / Scalability / i18n	Out of scope	Explicit non-goals (no multi-region HA, no i18n framework)
Maintainability	Accepted	Specs + Test Coverage Map + npm test as process bar

The table shape

Every product keeps one main table in [quality-attributes.md](#):

Attribute	Target	Approach	How we verify	Status	Links
Security	...	...	...	Accepted	ADR, rules, ...

Column principles

Column	Write this	Avoid
Attribute	Standard ility name (Security, Performance, ...)	Vague "Quality" or feature names ("Lists")
Target	Prefer a number or countable bar (100%, 0, p95 < 200 ms, ≤ 2 clicks)	"Make it fast" with no measure
Approach	How the bar is realized or why it is limited / out of scope	A second Target paragraph
How we verify	Tests, manual check, or N/A	"Somehow"
Status	One of the four values below	Mixing Accepted wording with Deferred intent
Links	ADR / rule / code path / feature pointer, or —	Orphan Accepted rows with no enforcement link

Teaching note: Targets on **Deferred** / **Out of scope** rows are often **illustrative classroom examples**, not production SLOs. Only **Accepted** (and feature FR/SC) constrain implementation by default.

Status values — principles

Status	Meaning	When writing / coding
Accepted	In force for this product	Do not regress; back with Links (ADR/rule/tests)
Accepted (minimal)	Thin bar in force	Meet the stated Approach only; do not expand into a platform
Deferred	Documented backlog / learning example	Guidance only — implement only if a feature or human requires it
Out of scope	Explicit non-goal	Do not invent HA, i18n frameworks, etc.

Choosing Status

1. **Can you verify it today** (automated or documented manual gate)? → candidate for **Accepted** or **Accepted (minimal)**.
2. **Useful to teach or plan, but not enforced?** → **Deferred** (keep a numeric Target as an example).
3. **We will not build for this?** → **Out of scope** (say so in Approach).
4. **Promoting Deferred → Accepted** requires matching **How we verify** — do not flip Status without tests or a real manual gate.

Links column — principles

Link type	Use when
ADR (docs/adr/...)	<i>Why</i> this Approach (e.g. ADR-0002 for Security)
Cursor rule (.cursor/rules/...)	<i>How</i> agents must behave every day
Code path	Minimal concrete implementation (e.g. logger.js)
Feature / framework	Process or FR/SC that carries the bar
—	Common for Deferred rows with no artifact yet

When Approach changes → update or add an ADR and refresh **Links**.

When agents need a lasting constraint → add/update a Cursor rule and link it.

Principles for writing good NFR rows

1. **One ility per row**. Don't combine Security and Performance into one Attribute.
2. **Prefer measurable Targets**. Numbers beat adjectives.
3. **Separate Target from Approach**. Target = bar; Approach = how/limits.
4. **Be honest about Status**. Deferred numbers are not secret Accepted gates.
5. **Link enforcement for Accepted rows**. An Accepted Security row without ADR/rule/tests is a smell.
6. **Keep feature-local quality in features**. Validation copy, one-screen UX notes → FR/SC + Gherkin.
7. **Out of scope is a feature, not an omission**. Explicit non-goals stop agents from inventing multi-region HA or i18n.
8. **Accepted (minimal) means thin**. Observability via Winston logs ≠ "build Datadog."
9. **Align with ADRs and rules**. Don't contradict ADR-0002 in the Security Approach.
10. **Update the table when reality changes**. New auth model, new logging, or a promoted bar → edit the row in the same change set when possible.

Feature-local NFRs

If only one feature needs a bar:

1. Put it under that feature's **Requirements (FR-00N)** and/or **Success Criteria (SC-00N)**.
2. Add Gherkin when tests must prove it.
3. Optionally add a one-line pointer on a related app-wide row ("see Feature N **FR-00N** / **SC-00N**").

Do **not** invent a new feature file solely to "add performance."

Workflow: add or change a bar

1. Edit [quality-attributes.md](#) — **Target, Approach, How we verify, Status, Links**. Prefer a number in **Target**.
 2. If **Approach** changes → [write/update an ADR](#); put it in **Links**.
 3. If agents need a lasting coding constraint → Cursor rule; put it in **Links**.
 4. If API/schema/rules change as a result → `features/reference/` in the same feature PR.
 5. **Deferred** → **Accepted** only with verification that matches **How we verify**.
-

Checklist (before you call a row "done")

- ☐ Attribute name is a clarity (not a feature title)
 - ☐ Target is measurable (or explicitly qualitative with a clear bar)
 - ☐ Approach explains realization **or** deliberate limit / non-goal
 - ☐ How we verify is realistic for the Status
 - ☐ Status matches enforcement reality (Accepted ↔ Links / tests)
 - ☐ Links point at ADR / rule / code / feature — or — on purpose
 - ☐ No conflict with existing Accepted ADRs or Cursor rules
 - ☐ Feature-only concerns left in FR/SC, not forced into the app-wide table
-

Anti-patterns

Avoid	Do instead
Treating every Deferred Target as a sprint commitment	Honor Status; only Accepted (+ feature FR/SC) constrain by default
"Make it secure" with no Target	100% protected routes auth'd; 0 cross-user leaks in tests
Accepted row with Links = — forever	Add ADR/rule/tests or demote to Deferred
Putting "user can create a list" in NFRs	Feature story / FR
Inventing i18n or HA because a Target number looks cool	Respect Out of scope
Duplicating the whole NFR table inside every feature	Link here; keep feature-local bars in FR/SC

How this fits the other guides

Product behavior (what)	→ features/writing-feature-requirements.md features/writing-feature-design.md
Architecture why	→ docs/adr/writing-adrs.md
Quality bars (how good)	→ docs/nfr/writing-quality-attributes.md (this file)
Day-to-day how for agents	→ .cursor/rules/*.mdc (incl. quality-attributes.mdc)

Part 4: Architecture Diagrams (C4)

Architecture diagrams (C4)

C4 views for the **Todo** example application in **OC CS Speckit**, as Mermaid. Source of truth for *why* the shape exists: [ADR-0001](#).

File	C4 level	Shows
c4-context.md	Context	People and systems
c4-container.md	Container	SPA, API, MySQL
c4-component-backend.md	Component	Backend API layers
c4-component-frontend.md	Component	Frontend SPA layers
c4-deployment.md	Deployment	User PC (browser) + Web Server (Apache, Node, MySQL)

Preview in Cursor

1. Wrap must be a ````mermaid` fence with `C4Context` / `C4Container` / `C4Component` / `C4Deployment` on the first line.
2. Use **Markdown Preview Mermaid Support** (or another C4-capable Mermaid preview) — Cursor’s default preview often does not render C4.
3. Command Palette → open that extension’s preview (not only the built-in one if they conflict).

PDF

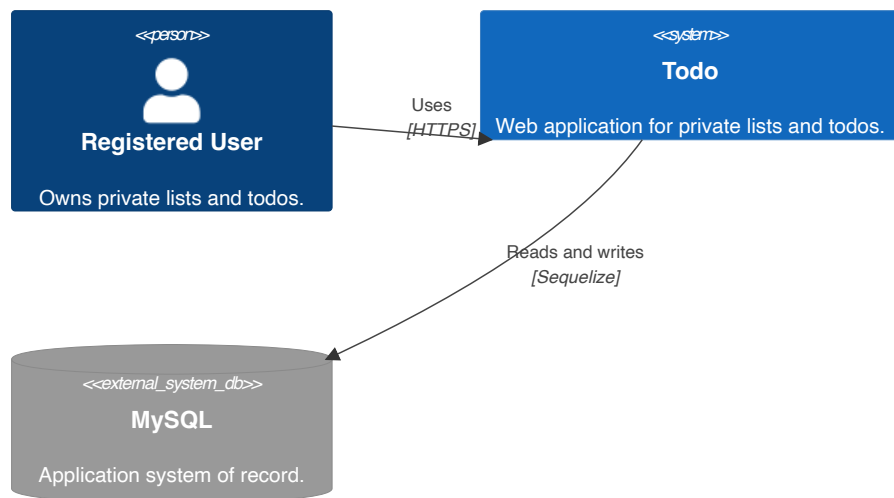
`npm run specs:pdf` **includes** this folder (Part 4) and **renders Mermaid** (including C4) via `md-mermaid-pdf` with a bundled Mermaid build (works offline). Layout quality matches Mermaid C4 limits — same as preview.

Adding a new `docs/arch_diagrams/*.md` file is enough; preferred order is listed in `scripts/export-specs-pdf.mjs` (`ARCH_DIAGRAM_ORDER`).

C4 Level 1 — System context

Todo (the OC CS Speckit example application) stores each registered user's private lists and todos in MySQL through a server API. There are no external SaaS dependencies.

System Context — Todo



Notes

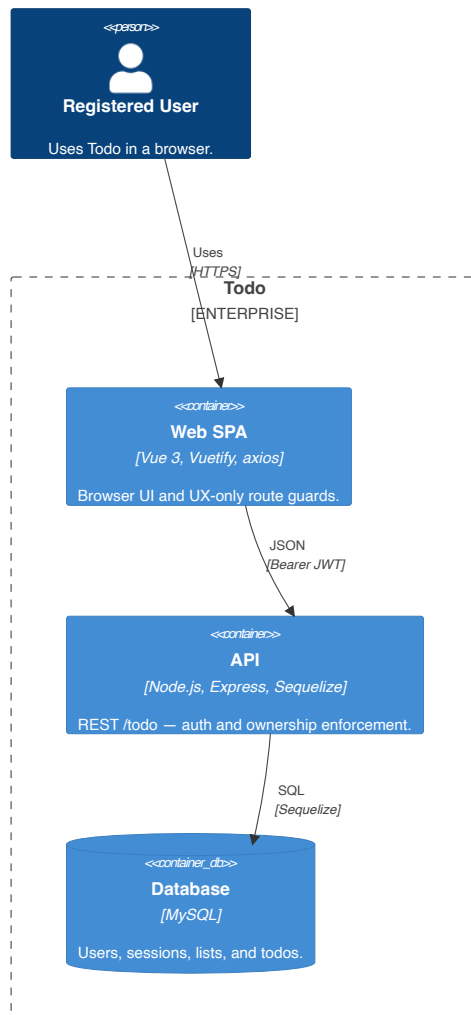
- The Todo system contains the Vue SPA and Express API; the [container diagram](#) expands that boundary.
- The API is the source of truth. Browser storage is only a session/UX hint.

Related: [ADR-0001](#) · [ADR-0003](#)

C4 Level 2 — Containers

Monorepo split: browser SPA talks to a stateless REST API; API owns auth and `userId` scoping; MySQL holds rows.

Container Diagram — Todo



Notes

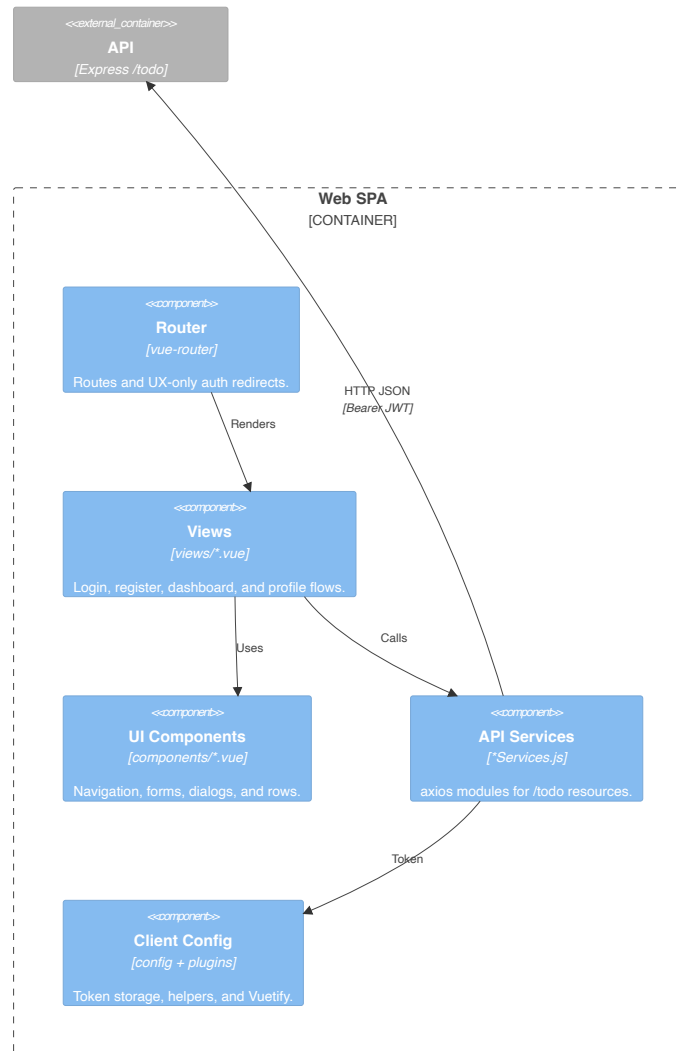
- API routes are mounted under `/todo/` ; authenticated requests carry a Bearer JWT backed by the Session table.
- The API assigns and scopes ownership from `req.user.id` ; the browser never supplies a trusted owner ID.
- Dev ports: frontend `8082` · backend `3200` ; CORS origin must match the SPA.

Related: [project-structure.mdc](#) · [api.md](#)

C4 Level 3 — Frontend components

Vue SPA inside `frontend/src/`, ordered along the user interaction and API request path.

Component Diagram — Web SPA



Notes

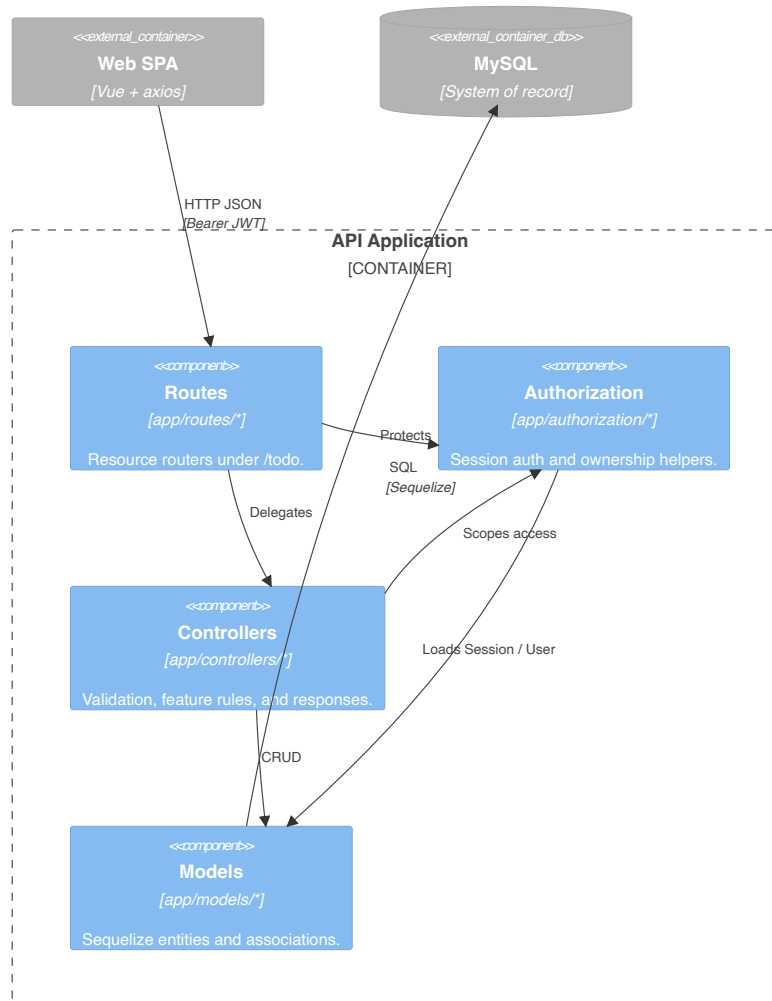
- Router guards and `localStorage` improve UX only; the API remains authoritative.
- Views may compose UI components that call services for dialog actions; the main request spine is simplified above.
- API modules follow the `*Services.js` naming rule.

Related: [frontend-services.mdc](#) · [ui-style-system.mdc](#)

C4 Level 3 — Backend components

Express app inside `backend/` , ordered along the HTTP handling path.

Component Diagram — API



Notes

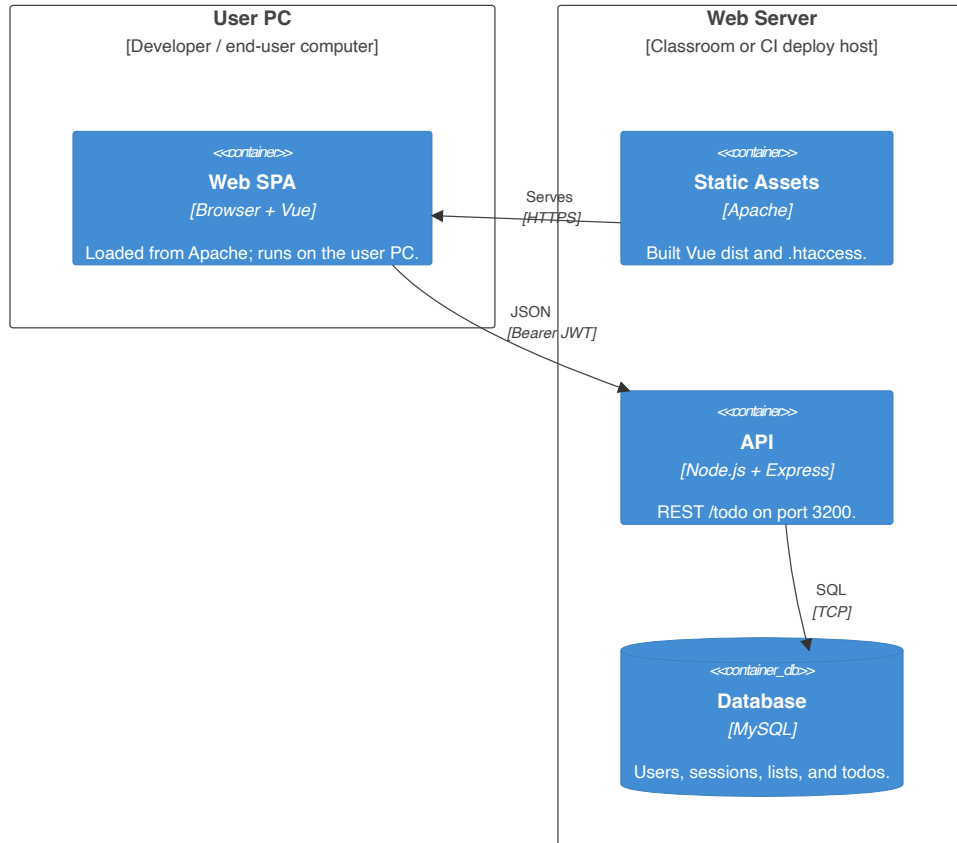
- Protected routes authenticate first; controllers reuse authorization helpers for ownership checks.
- Cross-user resources return `404` , while missing or invalid sessions return `401` .
- Database/auth configuration and Winston logging are omitted to keep the request path readable; they remain under `app/config/` .

Related: [ADR-0002](#) · [auth-patterns.mdc](#) · [security.mdc](#)

C4 Level 4 — Deployment

Logical deployment: the **User PC** runs the SPA in a browser; the **Web Server** hosts static assets, the Node API, and MySQL.

Deployment Diagram — Todo



Typical ports

Location	Service	Port
User PC	Browser	—
Web Server	Apache (SPA)	80 / 443
Web Server	Backend API	3200 (or reverse-proxied)
Web Server	MySQL	3306

Notes

- Nested browser/Apache/runtime/database deployment nodes are intentionally flattened because Mermaid C4 packs deep nesting poorly in PDFs.
- **Local XAMPP classroom:** User PC and Web Server may be the **same** physical machine; the diagram preserves the logical browser/server boundary.
- **CI deploy** (`.github/workflows/deploy.yml`): builds SPA + backend, SSH-deploys static files and Node app to the Web Server; DB credentials via secrets.

Related: [ADR-0001](#) · [c4-container.md](#) · `.github/workflows/deploy.yml`

Part 5: Feature Specifications

README.md

Feature Specifications

Spec-driven development (SDD) source of truth for **OC CS Speckit** (Todo is the example application in this repo).
No application code may be written unless it maps to a requirement in one of these files.

Methodology: [framework.md](#) — how to write, trace, and ship feature specs.
Student guide (requirements): [writing-feature-requirements.md](#) (PDF) — stories, FRs, initial data model, Gherkin AC.
Student guide (design): [writing-feature-design.md](#) (PDF) — ownership, API, screens, test map, DoD, out of scope.
Student guide (living reference): [reference/writing-living-reference.md](#) (PDF) — update api / data-model / behavior in the same PR.

Regenerate writing-guide PDFs: `npm run writing-guides:pdf`

Sprints (timeboxes, iterations, team planning) live in your agile tool — they are **not** part of these specs. One sprint may contain multiple features; one feature may span sprints. Specs describe **what** to build; sprints describe **when** the team works on it.

Feature catalog

ID	File	Branch	Status	Depends on
1	feature-1-user-auth.md	feature/1-user-auth	Ready	—
2	feature-2-todo-list-management.md	feature/2-todo-list-management	Ready	Feature 1
3	feature-3-todo-list-item-management.md	feature/3-todo-list-item-management	Ready	Features 1–2
4	feature-4-user-profile-management.md	feature/4-user-profile-management	Ready	Features 1–3
5	feature-5-todo-due-date.md	feature/5-todo-due-date	Ready	Features 1–3

Branch roles: `main` = scaffold-only starter kit · `dev` = integration (branch from `main` , merge features here) · `feature/N-*` = feature implementation (branch from `dev`).

Implement features in dependency order (1 → 2 → 3; 4 and 5 after 3). Features 4 and 5 do not depend on each other.

Living reference (current integrated state)

Keep these snapshots in sync with the codebase when schema or API changes — **in the same PR as implementation** (required DoD; see [Merge checklist + Agility sync](#)). Use each feature’s **Agent implementation request** block when implementing with Cursor.

File	Purpose
reference/README.md	How to maintain reference docs
reference/writing-living-reference.md	Student guide — writing/updating living reference
reference/data-model.md	Current database tables and associations

reference/api.md	Current REST API under /todo/
reference/behavior.md	Current product rules (ownership, sort, validation, UI)

New features: follow the template in [framework.md](#) — **Status**, **Input**, story **Priority** / **Independent test**, **FR-00N**, **Assumptions**, **Edge Cases**, **SC-00N**, **Key Entities**, plus **Agent implementation request** and **Definition of Done**.

Feature specs define **changes**; reference files describe **what exists now**. Spec evolution after merge: prefer a new feature delta — [framework.md](#).

Implementation order (each feature)

Cursor: Implement @features/feature-N-...md per its Agent implementation request and Definition of Done. (or layer-by-layer — see [framework.md](#)).

1. Backend models and associations
2. Backend routes, controllers, authorization helpers
3. Backend tests (Jest + supertest)
4. Frontend services (*Services.js , axios client)
5. Frontend views and components
6. Frontend tests (Vitest + @vue/test-utils)
7. Router updates and manual verification

Related project docs

- Cursor rules: `.cursor/rules/`
- Architecture decisions: `docs/adr/` — [index](#)
- Quality attributes (NFRs): `docs/nfr/` — [index](#)
- Living reference: `features/reference/` (data model + API + **behavior/rules** snapshot on `dev`)
- Backend env: `backend/.env` (copy from `backend/.env.example`)
- Test env: `backend/.env.test` (copy from `backend/.env.test.example`)
- UI references (optional): `docs/ui/` — link Figma exports from each feature spec

Running tests

```
npm test           # from repo root – backend + frontend
npm run test:backend # Jest
npm run test:frontend # Vitest
```

Export specs to PDF

Product specs only (ADRs, NFRs, C4, features — no rules/guides/reference):

```
npm run specs:pdf:app
```

Output: `docs/todo-app-specs.md` · `docs/todo-app-specs.pdf`

Full methodology pack (rules + ADRs + NFRs + diagrams + specs + reference):

```
npm install          # once — installs md-mermaid-pdf at repo root
npm run specs:pdf
```

If PDF generation fails with a missing Chrome error, either use installed Google Chrome / Edge (macOS, Windows, or Linux paths are detected) or run the one-time browser download:

```
npm run specs:pdf:setup
npm run specs:pdf
```

Output:

- docs/oc-cs-speckit-specs.md — combined Markdown (rules, ADRs, specs, reference)
- docs/oc-cs-speckit-specs.pdf — PDF export

Included in **specs:pdf** (auto-discovered each run):

1. All .cursor/rules/*.mdc (preferred order, then any extras alphabetically)
2. docs/adr/README.md + every docs/adr/NNNN-*.md (numeric order)
3. docs/nfr/README.md + docs/nfr/*.md (quality attributes)
4. docs/arch_diagrams/*.md (C4 / architecture diagrams; Mermaid rendered in PDF)
5. features/README.md , features/framework.md
6. Every features/feature-N-*.md (numeric order)
7. features/reference/*.md

Adding a new feature, ADR, NFR, or arch-diagram markdown file is enough — no edits to scripts/export-specs-pdf.mjs . Mermaid fences (including C4) render in the PDF via md-mermaid-pdf .

Manual alternative (no npm script):

```
# Prefer npm run specs:pdf — it strips .mdc frontmatter, discovers new files, and renders Mermaid.
npx md-mermaid-pdf docs/oc-cs-speckit-specs.md
```

Note: a plain md-to-pdf run leaves Mermaid as code blocks; npm run specs:pdf renders diagrams.

framework.md

Spec-Driven Development Framework

How **OC CS Speckit** writes, traces, and ships **feature specifications** (illustrated here by the Todo example application). This document is the methodology handbook; individual feature files are the requirements.

Related: [Feature catalog](#) · [ADRs](#) · [Quality attributes \(NFRs\)](#) · [Living reference](#) · [Constitution](#)

Purpose

Spec-Driven Development (SDD) inverts the usual order: **spec first, code second, tests as proof**.

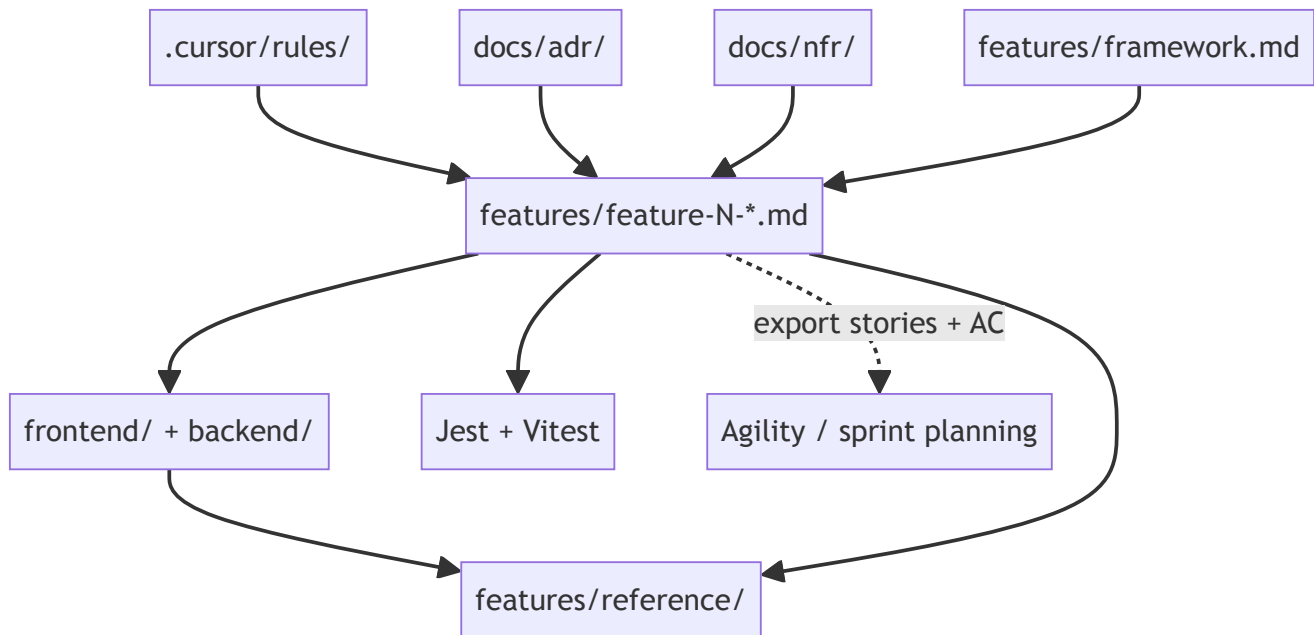
Role	Responsibility
Feature specs (features/feature-N-*.md)	Authorize <i>what</i> to build
Cursor rules (.cursor/rules/)	Constrain <i>how</i> to build (stack + agent-behavior.mdc)
Tests (Jest, Vitest)	Verify spec + implementation match
Reference docs (features/reference/)	Snapshot <i>what exists now</i> on dev (API, schema, behavior/rules)
Quality attributes (docs/nfr/)	App-wide <i>ilities</i> / NFR bars — see also quality-attributes.mdc (Accepted vs Deferred literacy)
ADRs (docs/adr/)	Record <i>why</i> cross-cutting architecture choices were made
Sprints / timeboxes (Agility, Jira, etc.)	Plan <i>when</i> work happens — outside these specs

No application code may be written unless it maps to an explicit requirement in a feature file (see constitution Principle 1).

Artifact map

.cursor/rules/	← stack conventions (how)
docs/adr/	← architecture decisions (why)
docs/nfr/	← quality attributes / ilities (bars)
features/framework.md	← this file (process)
features/feature-N-*.md	← product requirements (what)
↓	
frontend/ + backend/	← implementation
↓	
tests/	← verification

↓
features/reference/ ← integrated snapshot; updated in feature PR when API/schema/rules change



Specs define changes. Reference files describe the current integrated product. **NFRs** in `docs/nfr/` record app-wide quality bars; feature-local bars go in **Requirements (FR-00N)**. **Sprints** assign stories to iterations in your agile tool; they are not fields in feature markdown.

Feature spec template

Every new feature uses `features/feature-N-short-name.md` with these sections **in order**. Copy [feature-1-user-auth.md](#) as the canonical example.

Header

```
# Feature: <Human-readable title>

**Feature ID:** N
**Branch pattern:** `feature/N-short-name`
**Status:** Draft | Ready | Shipped
**Created:** YYYY-MM-DD
**Input:** One-line intent – what prompted this feature (the user description)
**Depends on:** [Feature X – ...](feature-X-...md), ... ← omit if none
**Related:** `features/reference/...`, [ADR-NNNN](../docs/adr/NNNN-title.md) ← optional
```

- **Feature ID** — sequential integer; never reuse a retired ID.
- **Branch pattern** — one Git branch per feature, branched from `dev`, merged back to `dev` (never `main`).
- **Status** — `Draft` while writing; `Ready` when FR/Gherkin complete; `Shipped` after merge to `dev`.
- **Input** — mirrors GitHub Spec Kit's feature description (what/why, not stack).
- **Depends on** — link to feature files whose code must already be on `dev`.

Required sections (in order)

Section	Purpose
User Stories	US–N.n with Priority , Independent test , link to Gherkin
Requirements → Functional Requirements	Numbered FR–00N rules (was “System Requirements”)
Assumptions	Explicit dependencies and scope boundaries
Edge Cases	Boundary/error cases not covered in a single scenario
Success Criteria	Numbered SC–00N measurable outcomes for this feature
Data Ownership & Isolation	Multi-user scope rules (when applicable)
Key Entities	Conceptual data — what exists, relationships (no column types)
API Requirements	Endpoints, payloads, status codes (if applicable)
Screen Requirements	Routes, views, UX (if applicable)
Data Model Requirements	Tables, columns, associations (if applicable)
Acceptance Criteria (Gherkin)	Testable Given / When / Then scenarios
Test Coverage Map	Each scenario → test file / area
Agent implementation request	Cursor prompt block (includes reference updates)
Definition of Done	Merge checklist for this feature
Out of Scope	Explicit deferrals with links to other feature files

Optional when needed: **Delivered to Feature X** (handoff notes). App-wide ilities live in [docs/nfr/quality-attributes.md](#) — do not duplicate the full table in every feature.

User story format

```
### US–N.1: Short title
**As a** ...
**I want to** ...
**So that** ...

**Priority:** P1
**Independent test:** How this story can be verified alone (e.g. "Register via API and reach protected home")
**Acceptance scenarios:** see ### US–N.1 under Acceptance Criteria
```

Use **As a** or **As the** (for system-level stories). Number stories **US–<feature-id>.<story-number>** — e.g. Feature 2’s third story is US–2.3 . Story numbers restart at **.1** in each feature file. **Priority:** **P1** = must ship in this feature; **P2** = important; **P3** = nice-to-have.

Functional requirements format

Rename the old **System Requirements** section. Number every rule:

Requirements

Functional Requirements

- ****FR-001****: System MUST ...
- ****FR-002****: Users MUST be able to ...

Restart `FR-001` in each feature file. While drafting, mark unknowns: `[NEEDS CLARIFICATION: ...]` — resolve before `Status: Ready` .

Assumptions, edge cases, success criteria

Assumptions

- Feature 1 auth is on `dev`
- ...

Edge Cases

- Empty required field → `400` or client validation block
- Cross-user resource → `404`

Success Criteria

- ****SC-001****: Every Gherkin scenario has at least one automated test before merge
- ****SC-002****: ...

Feature-local **SC** items only; app-wide bars stay in [docs/nfr/quality-attributes.md](https://docs.nfr/quality-attributes.md).

Key Entities

Conceptual model before **Data Model Requirements** (GitHub Spec Kit alignment):

Key Entities

- ****User****: account owner; has many lists and todos
- ****List****: named group of todos; belongs to one user

No column types here — those belong in **Data Model Requirements** or `features/reference/data-model.md` .

Gherkin format

Group scenarios under a `###` heading that includes the **story ID** and title (same as the user story):

US-1.1 – Registration

Scenario: Descriptive name

```
* **Given** ...
* **When** ...
* **Then** ...
* **And** ...
```

Each `### US-N.n` block under **Acceptance Criteria** owns the scenarios for that user story. One story may have many scenarios; do not mix scenarios from different stories under one heading.

Every scenario must appear in the **Test Coverage Map** and have at least one automated test before the feature is done.

GitHub Spec Kit alignment

This repo uses one merged `feature-N-*.md` per capability (Todo example app + fixed stack). [GitHub Spec Kit](#) splits **spec** (what/why) from **plan** (how). Map phases as follows:

Spec Kit phase	OC CS Speckit artifact
speckit.specify	User Stories, FR, Assumptions, Edge Cases, SC, Gherkin, Out of Scope
speckit.plan	ADRs, <code>.cursor/rules/</code> , API/Data/Screen sections in feature file
speckit.tasks	Test Coverage Map + layer order
speckit.implement	Agent implementation request + code
Post-merge snapshot	<code>features/reference/</code>

Spec Kit's `spec.md` avoids stack detail; our **API Requirements** and **Data Model Requirements** are intentional plan-level content in one file for traceability and Agility export.

Traceability

Spec artifact	Git	Tests	Agility export
Feature file	<code>feature/N-*</code> branch	—	Epic (Portfolio Item)
<code>US-N.n</code>	—	—	Story
<code>#### Scenario:</code>	—	Jest / Vitest <code>it("...")</code>	Test (acceptance criteria)
Stable refs	—	—	<code>TS-F{N}-US{N}. {n}</code> , <code>TS-F{N}-AC{nnn}</code>

Export backlog: `npm run agility:export` or `npm run agility:push` (see [docs/agility-import/README.md](#)). Feature files matching `features/feature-N-*.md` are auto-discovered; epic titles come from `# Feature: ...`.

Test traceability

Tests must link back to the spec in three layers:

```
feature-3-todo-list-item-management.md
└─ US-3.1 – Add tasks to a list
```

```
└─ Scenario: User adds a todo to the selected list
   └─ backend/tests/todos.test.js → it("User adds a todo...")
```

File header

Every feature test file starts with:

```
/**
 * Feature 3 – Todo List Item Management
 * Spec: features/feature-3-todo-list-item-management.md
 */
```

Harness-only files (`app.test.js` , `App.test.js`) are exempt — they verify the test setup, not product behavior.

Nested `describe` blocks

```
describe("Feature 3 – Todo API", () => {
  describe("US-3.1 – Add tasks to a list", () => {
    it("User adds a todo to the selected list", async () => { /* ... */ });
    it("User adds a todo with an empty title", async () => { /* ... */ });
  });
});
```

- **Outer `describe`** — feature name (matches spec title).
- **Inner `describe`** — `US-N.n` + story title (matches AC `###` heading).
- **`it` name** — exact Gherkin **Scenario** title from the spec.

Test Coverage Map (in each feature spec)

The map is the authoritative index. Prefer this column layout:

Story	Scenario	Test file	Test name
US-3.1	User adds a todo to the selected list	backend/tests/todos.test.js	it("User adds a todo to the selected list")
US-3.1	User adds a todo with an empty title	frontend/tests/Dashboard.test.js	it("User adds a todo with an empty title")

Auditing coverage

```
# Find all tests for a story
rg "US-3.1" features/ backend/tests frontend/tests

# Find a scenario across spec and tests
rg "User adds a todo with an empty title" features/ backend/tests frontend/tests
```

Every `#### Scenario` in the spec must have ≥ 1 matching `it` . Every feature `it` must trace to a scenario.

Architecture Decision Records (ADRs)

ADRs live in [docs/adr/](#) and answer **why** — not **what** (feature specs) or **how** (Cursor rules).

Write an ADR when...	Use instead...
Choosing client vs server, auth model, DB strategy	Feature spec for product behavior
Documenting tradeoffs and rejected alternatives	Cursor rule for ongoing patterns
A decision spans multiple features	Reference doc for current API/schema snapshot

Workflow: propose ADR → set status `Accepted` → link from affected feature headers → encode outcome in `.cursor/rules/` if it becomes a pattern.

See [docs/adr/README.md](#) for the template and index.

Workflow per feature

1. Write or update the spec first

Add `features/feature-N-...md` before implementation. If behavior is not in the spec, do not implement it (or update the spec first).

2. Branch from `dev`

```
git checkout dev && git pull
git checkout -b feature/N-short-name
```

3. Implement in layer order

1. Backend models and associations
2. Backend routes, controllers, authorization helpers
3. Backend tests (Jest + supertest)
4. Frontend services (`*Services.js` , axios client)
5. Frontend views and components
6. Frontend tests (Vitest + `@vue/test-utils`)
7. Router updates and manual verification

Work in **atomic steps** — one layer per commit when possible (constitution Principle 4).

4. Map tests to Gherkin

Fill the **Test Coverage Map** as you add tests. No `expect(true).toBe(true)` ; use edge cases from the spec.

5. Merge to `dev`

When every user story is implemented and every Gherkin scenario has a test, open a PR (or merge locally) using the [Merge checklist + Agility sync](#).

```
git checkout dev && git merge feature/N-short-name
```

6. Update living reference (required when integrated product changed)

In the **same PR as the implementation** (preferred — include in the [Agent implementation request](#) block):

If you changed...	Update
Tables, columns, associations	reference/data-model.md
Routes or payloads	reference/api.md
Product rules (ownership, sort, validation, UI rules embedded in code)	reference/behavior.md

Skipping reference updates is the most common SDD drift failure — treat as Definition of Done, not optional cleanup.

When to add vs edit

Situation	Action
New capability	New feature-(N+1)-...md + row in README catalog
Cross-cutting architecture choice	New docs/adr/NNNN-...md + link from feature specs
Clarify unmerged spec	Edit the feature file in place
Change already on dev	Follow Spec evolution after merge
"What exists now?" (API / schema / rules)	Update features/reference/ (api.md, data-model.md, behavior.md) — not the feature spec alone
Minor UI polish (colors, spacing, button labels on an existing screen)	See Minor UI changes — usually edit rule + Screen Requirements, not a new feature

Feature specs are **deltas**; reference docs are **current state**.

Spec evolution after merge

Once a feature has merged to `dev`, treat its spec as **released history** unless the team explicitly amends it.

Policy	Rule
Prefer a new delta	New behavior, bugfix with new AC, or API/schema change → new feature-(N+1)-...md (or the next free ID) that describes only the change. Link Depends on / Out of Scope to the earlier feature.
Amend only if unreleased	Edit an existing feature-N-*.md in place when the change has not shipped to dev yet, or the team agrees the edit is a clarification (typo, clearer Gherkin) with no behavior change.
Do not rewrite history silently	Do not expand Feature 3's stories after Feature 5 has shipped to "fold in" new scope — that breaks traceability and Agility refs.
Agility after story/AC text changes	Re-export or <code>npm run agility:push -- --feature N --upsert</code> when user stories or Gherkin changed. Epic/story Reference IDs stay stable; names/descriptions update.

Reference always	Whether you add Feature N+1 or amend, update features/reference/ when the integrated API, schema, or product rules on dev changed.
------------------	---

Default: new feature file for post-merge product change; amend for pre-merge clarification only.

Merge checklist + Agility sync

Before merging `feature/N-*` → `dev` :

- ☐ **Spec** — Feature file matches what shipped (stories, **FR-00N**, **SC-00N**, Screen/API/Data, Out of Scope).
- ☐ **Tests** — Every Gherkin scenario has a real `it` ; Test Coverage Map complete; suites pass.
- ☐ **Living reference** — Update in this PR when the integrated product changed: [reference/data-model.md](#) (schema), [reference/api.md](#) (routes/payloads), and/or [reference/behavior.md](#) (product rules). Required DoD — not optional cleanup.
- ☐ **Catalog** — [features/README.md](#) row for new features; ADR links if architecture changed.
- ☐ **NFRs** — If an app-wide quality bar changed: update [docs/nfr/quality-attributes.md](#) (and ADR/rule if approach or coding constraint changed).
- ☐ **Agility sync** — If stories or Gherkin changed (or this is a new feature):
 - `npm run agility:export` and/or `npm run agility:push -- --feature N --upsert`
 - New `feature-N-*.md` files are **auto-discovered** (epic title = `# Feature: ...`) — no `FEATURE_FILES` edit
 - Or note “Agility deferred” with an owner

Human review: reject diffs that implement behavior not in the spec, or that skip reference updates when the API, schema, or product rules changed.

Minor UI changes

Small visual tweaks still follow SDD: update the **smallest artifact that authorizes the change**, then implement. You do **not** need a new feature file, ADR, or Agility re-export for polish alone.

Classify first

Kind	Examples	Update first
Design system (all screens)	Theme colors, typography, default button style, oc-cta, spacing scale	.cursor/rules/ui-style-system.mdc → <code>frontend/src/plugins/vuetify.js</code> (and <code>App.vue</code> for global classes)
Feature screen (one view/flow)	Button placement, dialog labels, empty-state copy, layout on dashboard	Screen Requirements in the owning features/ <code>feature-N-*.md</code>
Already specified	Spec already says “primary elevated button” — you are fixing layout to match	Code only
New behavior	New dialog, new field, changed validation or disabled rules	Feature spec + Gherkin + tests

Rule of thumb: if a future developer (or AI) could not infer the correct UI from existing docs, update a spec or rule before coding.

Workflow

1. **Classify** — design system vs feature screen vs already specified.
2. **Edit the smallest doc** — `ui-style-system.mdc` and/or **Screen Requirements** (often one to three bullets).
3. **Implement** — only the affected Vue files; follow [ui-style-system.mdc](#).
4. **Verify** — visual check in `npm run dev`; run `npm run test:frontend` when labels or interaction changed.
5. **Skip** — `features/reference/` (unless a product rule or API/schema changed), new ADRs, new feature files, Agility push unless story/AC text changed. If Screen Requirements change a stated UI rule (empty-state copy, overdue styling, where Log out lives), update [reference/behavior.md](#).

Three tiers (practical policy)

Tier	When	Artifacts
A — Design system	Change applies app-wide	<code>ui-style-system.mdc</code> → <code>vuetify.js</code> / <code>App.vue</code> → sweep components if needed
B — Feature screen	Change is local to one feature's UI	<code>feature-N-*.md</code> Screen Requirements → one or two <code>.vue</code> files
C — Cosmetic	Already implied by rules/spec (e.g. comfortable density)	Code only; note in commit message

When unsure, use **Tier B** — one bullet in Screen Requirements is cheap and keeps traceability.

What to test

Changed	Do
Colors / theme / global CSS class	Visual check + hard-refresh if HMR misses <code>App.vue</code> styles
Button or link label text	Update Screen Requirements; update Vitest if Gherkin names the label
Layout only	Visual check; tests optional
Dialog flow or validation	Gherkin + Vitest

Cursor requests (revise UI from spec)

After you update **Screen Requirements** and/or [ui-style-system.mdc](#), ask Cursor to implement from the spec — not from memory.

Request formula

```
Implement the UI changes in @features/feature-N-....md Screen Requirements
per @.cursor/rules/ui-style-system.mdc.
```

- Update only the affected Vue files (name them if you know them).
- Do not change API, backend, or behavior unless the spec says so.
- Run `npm run test:frontend` when labels or flows changed.

Core pattern: feature spec (what) + style rule (how) + scope guardrails.

What to @ -mention

Always	When relevant
features/feature-N-...md	ui-style-system.mdc
	Specific .vue file(s)
	testing-standards.mdc if tests must change

One feature file + the style rule is usually enough — you do not need to @ the whole repo.

Prompts by situation

Screen Requirements only (most common):

I updated Screen Requirements in @features/feature-2-todo-list-management.md.

Revise the dashboard sidebar UI to match the spec:

- + New List button placement and class oc-cta
- Dialog labels Create / Cancel as written

Follow @.cursor/rules/ui-style-system.mdc.

Files: frontend/src/views/Dashboard.vue only.

No API or test changes unless Gherkin scenario text changed.

Design system + feature spec:

Per @.cursor/rules/ui-style-system.mdc and
@features/feature-3-todo-list-item-management.md Screen Requirements,
apply the oc-cta button style to Add and align with Edit Profile.

Update Dashboard.vue and MenuBar.vue if needed.

Theme tokens stay in plugins/vuetify.js — no hex in components.

Gherkin labels or flows changed (needs tests):

@features/feature-4-user-profile-management.md was updated:

- Screen Requirements: Edit Profile dialog layout

Implement the UI delta in MenuBar.vue.

Update frontend/tests/MenuBar.test.js if any asserted text or selectors change.

Run npm run test:frontend.

One-liner:

Revise UI from @features/feature-N-...md Screen Requirements per @.cursor/rules/ui-style-system.mdc;
[file.vue] only; no API changes.

Guardrails (add to any prompt)

Phrase	Why
"Do not add behavior not in the spec."	Constitution Principle 1
"Do not create a new feature file."	Polish ≠ new capability
"No backend changes."	Keeps scope to UI
"Match exact button labels from Screen Requirements."	Traceability to Gherkin
"Use oc-cta for primary labeled CTAs; no buttons inside v-card-title."	ui-style-system.mdc

After Cursor finishes

1. Visual check in `npm run dev` (hard-refresh if global CSS changed).
2. `npm run test:frontend` if labels, dialogs, or flows changed.
3. Confirm the diff matches **Screen Requirements** — reject extra polish the spec does not authorize.

Do not

Avoid	Do instead
Code first, spec later	Update rule or Screen Requirements first
New Feature N+1 for polish	Design system + existing feature screen sections
Put all UI guidance in a loose <code>design.md</code> with no rule	Use <code>ui-style-system.mdc</code> so Cursor applies globs on <code>frontend/**</code>
Nest labeled CTAs inside <code><v-card-title></code>	Use <code><v-card-item> #append</code> — title typography skews button label size

Optional visual references: export frames to `docs/ui/feature-N/` and link from **Screen Requirements** or the feature header **Related** line. Specs remain the functional source of truth; images are guidance only.

Agent implementation request

Every feature file ends with an **Agent implementation request** section: a copy-paste prompt block Cursor reads when you `@` the spec and ask to implement the feature. It bundles **living reference** updates so they are not forgotten at merge.

Place it after **Test Coverage Map**, before **Definition of Done** and **Out of Scope**.

Template (add to each `feature-N-*.md`)

Agent implementation request

Copy when asking Cursor to implement this feature (`@`` this file):

```
\\\`text
Implement Feature N from @features/feature-N-short-name.md on branch `feature/N-short-name`.
```

Follow layer order in `@features/framework.md` (models → routes → backend tests → frontend → frontend tests).

```
Map every Gherkin scenario in the Test Coverage Map; run `npm test` before finishing.
If API routes, payloads, schema, or product rules changed per this spec, update
@features/reference/api.md, @features/reference/data-model.md, and/or
@features/reference/behavior.md in the same PR to match shipped code.
Complete Definition of Done and the merge checklist in @features/framework.md.
Do not implement behavior not in this spec.
\\`\\`

**Reference updates for this feature:** list `api.md`, `data-model.md`, `behavior.md`, or `none`
(no integrated-product delta).
```

Customize the last line per feature. Omit the reference-update sentence only when the feature has no **API Requirements**, no **Data Model Requirements**, and no new/changed **FR** product rules.

Cursor requests (implement full feature)

When starting work, paste the block from the feature file (or use this formula):

```
Implement Feature N from @features/feature-N-short-name.md on branch `feature/N-short-name`.

Follow @features/framework.md layer order and Test Coverage Map.
When API, schema, or product rules change, update @features/reference/api.md,
@features/reference/data-model.md, and/or @features/reference/behavior.md in the same PR.
Run `npm test`. Complete Definition of Done in the feature file.
```

One-liner:

```
Implement @features/feature-N-short-name.md per its Agent implementation request and Definition of Done.
```

Layer-by-layer prompts are still fine (e.g. “implement Data Model Requirements only”) — add reference updates on the **last** slice that touches API, schema, or product rules.

Definition of Done template

```
## Definition of Done

* [ ] Backend and frontend implemented per this spec (**FR-00N** satisfied)
* [ ] **Success Criteria (SC-00N)** met
* [ ] All mapped tests pass (`npm test`)
* [ ] Test Coverage Map complete
* [ ] `features/reference/data-model.md` updated (if schema changed)
* [ ] `features/reference/api.md` updated (if API changed)
* [ ] `features/reference/behavior.md` updated (if product rules changed)
```

Features vs sprints

	Feature spec	Sprint / iteration
--	--------------	--------------------

Question	What must the product do?	When will the team work on it?
Lives in	features/feature-N-*.md	Agility, Jira, etc.
Granularity	One file per shippable capability	Any grouping of stories
This repo	Source of truth for code + tests	Assigned after export/import

Example: Feature 2 (lists) and Feature 4 (profile) can ship in the same sprint, or Feature 3 can span two sprints — specs stay the same either way.

Adding Feature N+1 (checklist)

- ☐ Pick next sequential **Feature ID** and filename `feature-N-kebab-name.md`
- ☐ Fill header: branch pattern, **Depends on** links
- ☐ Complete user stories, **Requirements (FR-00N)**, **Assumptions**, **Edge Cases**, **Success Criteria (SC-00N)**, **Key Entities**, API/screen/data sections as applicable
- ☐ Write Gherkin acceptance criteria for all behavior
- ☐ Add **Test Coverage Map** before coding
- ☐ Add **Agent implementation request** (with correct reference file list) and **Definition of Done**
- ☐ Document **Out of Scope** with links to other features
- ☐ Add row to [features/README.md](#) catalog
- ☐ No Agility list edit required — `feature-N-*.md` is auto-discovered (`# Feature: ...` → epic name)
- ☐ Before merge: complete the [Merge checklist + Agility sync](#)

Cursor and AI usage

- Rules in `.cursor/rules/` apply automatically; they do not replace feature specs.
- In prompts, `@` -mention one feature file and **one slice** (e.g. “implement Data Model Requirements only”), or use the feature’s **Agent implementation request** block for full implementation.
- If the AI proposes behavior not in the spec, update the spec first or reject the change (constitution Principle 6).
- Living reference updates are part of the **Agent implementation request** — not a separate reminder after merge.

Anti-patterns

Do not	Do instead
Code first, spec later	Spec merge before or with first implementation PR
Put sprint numbers in feature files	Plan sprints in the agile tool
Skip Test Coverage Map	Map every Gherkin scenario before marking done
Change reference/ without a spec	Spec authorizes the delta; reference reflects merge
Merge API/schema/ rules change without updating reference/	Same PR (or immediate follow-up) — see merge checklist
Amend a shipped feature to add new scope	New feature-(N+1) delta — see Spec evolution after merge

Implement on main	feature/N-* → dev only
One giant “build the feature” prompt	Layer-by-layer micro-steps

Exports

Command	Output
<code>npm run specs:pdf:app</code>	ADRs + NFRs + C4 + feature specs → docs/todo-app-specs.pdf (no rules / writing guides / reference)
<code>npm run specs:pdf</code>	Rules + ADRs + NFRs + C4 diagrams + specs + reference → docs/oc-cs-speckit-specs.pdf (auto-discovers; Mermaid/C4 rendered)
<code>npm run agility:export</code>	CSV backlog for Agility Excel import (auto-discovers feature-N-*.md)
<code>npm run agility:push</code>	Push epics, stories, tests via Agility API
<code>npm run agility:push -- --feature N --upsert</code>	Update existing stories/tests for feature N; create missing
<code>npm run starter:zip</code>	Zip SDD starter kit for a new app — macOS/Windows/Linux (see docs/STARTER-KIT.md)

PDF and Agility both pick up new `features/feature-N-*.md` automatically. See [README](#) and [docs/agility-import/README.md](#).

feature-1-user-auth.md

Feature: User Authentication & Session Management

Feature ID: 1 **Branch pattern:** feature/1-user-auth **Status:** Ready **Created:** 2026-01-15 **Input:** Multi-user authentication and session management so each user can sign in and access private todo data **Related:** [ADR-0001 — Client-server multi-user architecture](#), [ADR-0002 — Security architecture](#)

User Stories

US-1.1: Register an account

As a new user

I want to create an account with my name, email, username, and password

So that I can sign in and manage my own private todo lists

Priority: P1

Independent test: Submit valid registration and land on protected home with `user` in `localStorage`

Acceptance scenarios: see ### US-1.1 under Acceptance Criteria

US-1.2: Sign in

As a registered user

I want to sign in with my username and password

So that I can access the application dashboard securely

Priority: P1

Independent test: Sign in with known credentials and receive session token + redirect to home

Acceptance scenarios: see ### US-1.2 under Acceptance Criteria

US-1.3: Stay signed in across page loads

As a signed-in user

I want my session to persist in the browser

So that I do not have to sign in again every time I refresh the page

Priority: P1

Independent test: Refresh or revisit protected route with valid `localStorage` session — no re-login

Acceptance scenarios: see ### US-1.3 under Acceptance Criteria

US-1.4: Sign out

As a signed-in user

I want to sign out

So that no one else can use my account on a shared device

Priority: P2

Independent test: Sign out clears server session and `localStorage` ; user lands on login

Acceptance scenarios: see ### US-1.4 under Acceptance Criteria

US-1.5: Block unauthenticated access

As the application

I want to require a valid session for all non-auth screens

So that users can only see and modify their own data

Priority: P1

Independent test: Navigate to protected route without session → redirect to login; API without token → 401

Acceptance scenarios: see ### US-1.5 under Acceptance Criteria

Requirements

Functional Requirements

- **FR-001:** Users MUST authenticate with **username + password** (not email-only login).
 - **FR-002:** Registration MUST collect first name, last name, email, username, and password.
 - **FR-003:** Passwords MUST be hashed with **bcrypt** (`SALT_ROUNDS = 10`) before persistence; hashes MUST never be returned by the API.
 - **FR-004:** Sessions MUST use a **JWT + Session table** pattern: token stored server-side; client sends `Authorization: Bearer <token>` .
 - **FR-005:** Session lifetime MUST be **24 hours** from creation.
 - **FR-006:** Login MUST reuse a non-expired session for the same user when one already exists.
 - **FR-007:** Default role for new users MUST be `worker` .
 - **FR-008:** Every authenticated request MUST resolve to exactly one user via `req.user.id` from the session token (foundation for Features 2–3 ownership).
 - **FR-009:** Registration MUST use shared `emailRules` from `frontend/src/config/validation.js` — required plus regex (`/^[^\\s@]+@[^\\s@]+\\. [^\\s@]+$/`); invalid format message: **"Enter a valid email address."**
-

Assumptions

- Greenfield app — no existing users or external identity provider.
- Single browser `localStorage` session per device (no multi-tab sync beyond shared storage).
- Lists and todos are deferred to Features 2–3; Feature 1 delivers auth and a minimal protected home placeholder only.

Edge Cases

- Duplicate username or email on register → 400 with clear message.
- Invalid login credentials → 401 (same message for wrong username or password).
- Missing or expired token on protected API → 401 ; frontend clears session and redirects to login.
- Whitespace-only required fields → rejected (client and/or server).

Success Criteria

- **SC-001:** Every Gherkin scenario in this feature has at least one automated test before merge.
 - **SC-002:** A new user can register, sign in, reach the protected home page, and sign out in one manual pass.
 - **SC-003:** `npm test` passes with backend auth and frontend router/register/login coverage.
-

Data Ownership & Isolation (foundation)

Feature 1 establishes identity; Features 2–3 enforce per-user data boundaries.

- Each user account is a separate tenant boundary for todo lists and items.

- No API in this feature returns another user's profile or session.
- Later features must never expose lists or todos across users — not in list responses, detail views, or error messages that confirm another user's resource exists.

API Requirements

Method	Endpoint	Auth	Purpose
POST	/todo/register	No	Create a new user account
POST	/todo/login	No	Authenticate and return session payload
POST	/todo/logout	Yes	Invalidate current session token

Login / register success response (flat JSON, no envelope):

```
{
  "userId": 1,
  "username": "jdoe",
  "email": "jdoe@example.com",
  "fName": "Jane",
  "lName": "Doe",
  "role": "worker",
  "token": "<jwt>"
}
```

Error response: { "message": "Human-readable explanation." } with appropriate HTTP status.

Screen Requirements

[View: Login Page] — route name `login`

- Full-screen auth layout (no `MenuBar`).
- Fields: username, password.
- Primary action: **Sign in** (`v-btn` , shows `:loading` while request is in flight).
- Link or button to navigate to registration.
- Inline error via `<v-alert type="error">` on failed login.

[View: Register Page] — route name `register`

- Full-screen auth layout (no `MenuBar`).
- Fields: first name, last name, email, username, password, confirm password.
- Email field uses shared `emailRules` from `frontend/src/config/validation.js` (required + regex format).
- Primary action: **Create account**.
- Link or button to navigate to login.
- Client-side validation before API call; server errors shown via `<v-alert type="error">` .

[View: Dashboard placeholder] — route name `home`

- Minimal protected landing page shown after successful login (full dashboard built in Feature 2).
- Displays a welcome message using the user's first name.
- **No `MenuBar`** in Feature 1 — auth pages and this placeholder use a full-screen layout only.

- **Sign out** button on this page (standalone `v-btn` ; removed from page content when `MenuBar` is added in Feature 2).

Key Entities

- **User**: registered account (name, email, username, role); owns future lists and todos.
- **Session**: server-side record tying a JWT token to a user; expires after 24 hours.

Data Model Requirements

users table

Field	Type	Rules
id	INTEGER PK	Auto-increment
fName	STRING	Required
lName	STRING	Required
email	STRING	Required, unique
username	STRING(100)	Required, unique; stored lowercase
password	STRING(255)	Required; bcrypt hash only
role	STRING(20)	Default worker

sessions table

Field	Type	Rules
id	INTEGER PK	Auto-increment
token	STRING	Required
email	STRING	Required
expirationDate	DATE	Required
userId	INTEGER FK	Required, references users.id

Acceptance Criteria (Gherkin)

US-1.1 — Registration

Scenario: User registers with valid information

- **Given** I am on the registration page
- **When** I enter valid first name, last name, email, username, password, and matching confirm password
- **And** I submit the form
- **Then** the API returns `201` with a user payload including `userId` , `username` , `email` , `token` , and `role`
- **And** my user record is stored in the database with a bcrypt password hash
- **And** I am redirected to the home page

- **And** my session is stored in `localStorage` under the key `user`

Scenario: User submits registration with missing email

- **Given** I am on the registration page
- **When** I leave the email field empty
- **And** I submit the form
- **Then** inline validation blocks the request
- **And** I see the message "Email is required."
- **And** no API request is sent

Scenario: User submits registration with invalid email format

- **Given** I am on the registration page
- **When** I enter a value that is not a valid email address (e.g. `notanemail`)
- **And** I submit the form
- **Then** inline validation blocks the request
- **And** I see the message "Enter a valid email address."
- **And** no API request is sent

Scenario: User submits registration with missing username

- **Given** I am on the registration page
- **When** I leave the username field empty
- **And** I submit the form
- **Then** inline validation blocks the request
- **And** I see the message "Username is required."
- **And** no API request is sent

Scenario: User submits registration with password too short

- **Given** I am on the registration page
- **When** I enter a password with fewer than 8 characters
- **And** I submit the form
- **Then** inline validation blocks the request
- **And** I see the message "Password must be at least 8 characters."

Scenario: User submits registration with mismatched passwords

- **Given** I am on the registration page
- **When** password and confirm password do not match
- **And** I submit the form
- **Then** inline validation blocks the request
- **And** I see the message "Passwords do not match."

Scenario: User registers with a duplicate username

- **Given** a user with username `jdoe` already exists
- **When** I submit registration with username `jdoe`
- **Then** the API returns `400` with `{ "message": "Username is already taken." }`
- **And** the error is displayed in a `<v-alert type="error">`

Scenario: User registers with a duplicate email

- **Given** a user with email `jane@example.com` already exists
 - **When** I submit registration with email `jane@example.com`
 - **Then** the API returns `400` with `{ "message": "Email is already registered." }`
 - **And** the error is displayed in a `<v-alert type="error">`
-

US-1.2 — Sign in

Scenario: User signs in with valid credentials

- **Given** I am on the login page
- **And** a registered user exists with username `jdoe` and a known password
- **When** I enter username `jdoe` and the correct password
- **And** I click **Sign in**
- **Then** the API returns `200` with a payload containing `userId`, `username`, `token`, and `role`
- **And** a session row is created or reused in the database
- **And** I am redirected to the home page
- **And** my session is stored in `localStorage` under the key `user`

Scenario: User signs in with invalid password

- **Given** I am on the login page
- **And** a registered user exists with username `jdoe`
- **When** I enter username `jdoe` and an incorrect password
- **And** I click **Sign in**
- **Then** the API returns `401` with `{ "message": "Invalid username or password." }`
- **And** I remain on the login page
- **And** the error is displayed in a `<v-alert type="error">`

Scenario: User signs in with missing username

- **Given** I am on the login page
- **When** I leave the username field empty
- **And** I click **Sign in**
- **Then** inline validation blocks the request
- **And** I see the message **"Username is required."**
- **And** no API request is sent

Scenario: User signs in with missing password

- **Given** I am on the login page
- **When** I leave the password field empty
- **And** I click **Sign in**
- **Then** inline validation blocks the request
- **And** I see the message **"Password is required."**
- **And** no API request is sent

US-1.3 — Stay signed in across page loads

Scenario: Signed-in user visits login page

- **Given** I have a valid session in `localStorage`
- **When** I navigate to the login page
- **Then** I am redirected to the home page

Scenario: API request includes session token

- **Given** I am signed in
- **When** the frontend makes an authenticated API request
- **Then** the request includes header `Authorization: Bearer <token>`

Scenario: Protected API request succeeds with a valid session

- **Given** I am signed in as user A

- **And** user B also exists
- **When** I send an authenticated `GET /todo/lists` request
- **Then** the API returns `200`
- **And** only lists owned by user A are returned

Scenario: Expired or invalid session token

- **Given** I am signed in with an expired or revoked token
- **When** the frontend makes an authenticated API request
- **Then** the API returns `401` with an unauthorized message
- **And** `localStorage` key `user` is cleared
- **And** I am redirected to the login page

US-1.4 — Sign out

Scenario: User signs out

- **Given** I am signed in on the home page
- **When** I click **Sign out**
- **Then** the API invalidates my session token on the server
- **And** `localStorage` key `user` is removed
- **And** I am redirected to the login page

US-1.5 — Block unauthenticated access

Scenario: Unauthenticated user accesses a protected route

- **Given** I have no session in `localStorage`
- **When** I navigate directly to the home page
- **Then** I am redirected to the login page

Test Coverage Map

Each scenario above must map to at least one automated test.

Story	Scenario	Test file	Test name
US-1.1	User registers with valid information	backend/tests/auth.test.js	User registers with valid information
US-1.1	User submits registration with missing email	backend/tests/auth.test.js	User submits registration with missing email
US-1.1	User submits registration with invalid email format	frontend/tests/Register.test.js	User submits registration with invalid email format
US-1.1	User submits registration with missing username	frontend/tests/Register.test.js	User submits registration with missing username
US-1.1	User submits registration with password too short	backend/tests/auth.test.js, frontend/tests/Register.test.js	User submits registration with password too short
US-1.1	User submits registration with mismatched passwords	frontend/tests/Register.test.js	User submits registration with mismatched passwords

US-1.1	User registers with a duplicate username	backend/tests/auth.test.js	User registers with a duplicate username
US-1.1	User registers with a duplicate email	backend/tests/auth.test.js	User registers with a duplicate email
US-1.2	User signs in with valid credentials	backend/tests/auth.test.js	User signs in with valid credentials
US-1.2	User signs in with invalid password	backend/tests/auth.test.js, frontend/tests/Login.test.js	User signs in with invalid password
US-1.2	User signs in with missing username	backend/tests/auth.test.js, frontend/tests/Login.test.js	User signs in with missing username
US-1.2	User signs in with missing password	backend/tests/auth.test.js, frontend/tests/Login.test.js	User signs in with missing password
US-1.3	Signed-in user visits login page	frontend/tests/router.test.js	Signed-in user visits login page
US-1.3	API request includes session token	backend/tests/authenticate.test.js	API request includes session token
US-1.3	Protected API request succeeds with a valid session	backend/tests/authenticate.test.js	Protected API request succeeds with a valid session
US-1.3	Expired or invalid session token	backend/tests/authenticate.test.js	Expired or invalid session token
US-1.4	User signs out	backend/tests/auth.test.js	User signs out
US-1.5	Unauthenticated user accesses a protected route	backend/tests/authenticate.test.js, frontend/tests/router.test.js	Unauthenticated user accesses a protected route

Agent implementation request

Copy when asking Cursor to implement this feature (@ this file):

Implement Feature 1 from @features/feature-1-user-auth.md on branch `feature/1-user-auth`.

Follow layer order in @features/framework.md (models → routes → backend tests → frontend → frontend tests).

Map every Gherkin scenario in the Test Coverage Map; run `npm test` before finishing.

If API routes, payloads, schema, or product rules changed per this spec, update @features/reference/api.md, @features/reference/data-model.md, and/or @features/reference/behavior.md in the same PR to match shipped code.

Complete Definition of Done and the merge checklist in @features/framework.md.

Do not implement behavior not in this spec.

Reference updates for this feature: `features/reference/data-model.md` , `features/reference/api.md` , `features/reference/behavior.md`

Definition of Done

- ☐ Backend and frontend implemented per this spec (**FR-00N** satisfied)
 - ☐ **Success Criteria (SC-00N)** met
 - ☐ All mapped tests pass (`npm test`)
 - ☐ Test Coverage Map complete
 - ☐ `features/reference/data-model.md` updated (if schema changed)
 - ☐ `features/reference/api.md` updated (if API changed)
 - ☐ `features/reference/behavior.md` updated (if product rules changed)
-

Out of Scope

- Password reset (`POST /todo/reset-password`)
- Email verification
- OAuth / social login
- Admin user management
- Full todo dashboard (Feature 2)

feature-2-todo-list-management.md

Feature: Todo List Management

Feature ID: 2 **Branch pattern:** feature/2-todo-list-management **Status:** Ready **Created:** 2026-02-01 **Input:** Signed-in users manage private named todo lists on one dashboard view; new lists are added via a dialog **Depends on:** [Feature 1 — User Authentication](#)

User Stories

US-2.1: Create todo lists

As a signed-in user

I want to create named todo lists (e.g. "Work", "Groceries")

So that I can organize tasks into separate groups

Priority: P1

Independent test: Open add-list dialog, create a list; it appears in the lists view

Acceptance scenarios: see ### US-2.1 under Acceptance Criteria

US-2.2: View my lists

As a signed-in user

I want to see all of my todo lists on one screen

So that I can see what groups I have created

Priority: P1

Independent test: Dashboard loads a single list of owned lists (no sidebar split)

Acceptance scenarios: see ### US-2.2 under Acceptance Criteria

US-2.3: Manage list rows

As a signed-in user

I want each list row to show **edit** and **delete** actions

So that I can manage lists without leaving the lists view (todo **items** open in a dialog — Feature 3)

Priority: P1

Independent test: Each list row exposes edit and delete icon actions

Acceptance scenarios: see ### US-2.3 under Acceptance Criteria

US-2.4: Rename and delete lists

As a signed-in user

I want to rename or delete a todo list

So that I can keep my workspace organized

Priority: P2

Independent test: Rename and delete an owned list from row actions; lists view updates

Acceptance scenarios: see ### US-2.4 under Acceptance Criteria

US-2.5: Private lists only

As a signed-in user

I want my lists visible only to me

So that other users cannot read or modify my list names

Priority: P1

Independent test: Cross-user list access returns `404` ; `GET /todo/lists` never returns another user's rows

Acceptance scenarios: see ### US-2.5 under Acceptance Criteria

Requirements

Functional Requirements

- **FR-001:** All list endpoints MUST require a valid session (`authenticate` middleware).
 - **FR-002:** A list MUST belong to exactly one user for its entire lifetime; ownership MUST never change.
 - **FR-003:** Every database read, update, and delete MUST include `userId: req.user.id` in the `where` clause.
 - **FR-004:** On create, `userId` MUST be set from `req.user.id` only — ignore or strip any `userId` in the request body.
 - **FR-005:** List names MUST be trimmed before save; empty strings MUST be rejected.
 - **FR-006:** Lists MUST be ordered alphabetically by name in API responses.
 - **FR-007:** This feature MUST deliver list CRUD and a **single-view** lists UI in `Dashboard.vue` (dialog-based add/edit/delete). No sidebar/main split. Todo **items** UI is Feature 3.
-

Assumptions

- Feature 1 auth and session handling MUST be merged to `dev` before implementing this feature.
- Lists and todos use **dialog-based** workflows (no split sidebar / main panel).
- `MenuBar` is introduced in this feature with basic sign-out (profile dropdown is Feature 4).

Edge Cases

- Empty or whitespace-only list name → client block and/or `400` .
- List name longer than 100 characters → `400` .
- Invalid `listId` → `400` ; unowned list → `404` .
- Unauthenticated dashboard or `GET /todo/lists` → redirect or `401` .

Success Criteria

- **SC-001:** Every Gherkin scenario has at least one automated test before merge.
 - **SC-002:** Signed-in user can create, view, rename, and delete lists on one screen without seeing another user's data.
 - **SC-003:** `npm test` passes for list API and dashboard lists-view behavior.
-

Data Ownership & Isolation

Each user owns their lists exclusively. Another authenticated user must not be able to view, rename, or delete them.

Rule	Requirement
Read scope	<code>GET /todo/lists</code> returns only lists where <code>userId = req.user.id</code> .
Write scope	<code>PUT</code> and <code>DELETE</code> apply only when the list row matches both <code>id</code> and <code>req.user.id</code> .

Create scope	New lists are always owned by the authenticated user.
Cross-user access	If a list belongs to another user, respond with 404 — never 403 (do not confirm the list exists).
UI scope	The lists view shows only lists returned by GET /todo/lists for the signed-in user.
Implementation	Use a shared helper (e.g. <code>getAccessibleListOrNull(req, listId)</code>) in <code>app/authorization/</code> — do not duplicate scope logic in controllers.

API Requirements

Method	Endpoint	Auth	Purpose
GET	/todo/lists	Yes	Fetch all lists for the authenticated user
POST	/todo/lists	Yes	Create a new list
PUT	/todo/lists/:listId	Yes	Rename a list
DELETE	/todo/lists/:listId	Yes	Delete a list owned by the caller

All endpoints return **only data owned by the authenticated user**. Cross-user access attempts return 404 .

Create list request body:

```
{ "name": "Groceries" }
```

List success response (200 / 201):

```
{
  "id": 1,
  "name": "Groceries",
  "userId": 42,
  "createdAt": "2026-07-02T12:00:00.000Z",
  "updatedAt": "2026-07-02T12:00:00.000Z"
}
```

Error response: { "message": "Human-readable explanation." } with appropriate HTTP status.

Not found / not owned: 404 (do not use 403).

Screen Requirements

[View: Application Dashboard] — route name home

Replaces the Feature 1 placeholder home page. **Single Vue view** (`Dashboard.vue`) — no sidebar / main-panel split.

Lists view (this feature)

- Heading: **My Lists**
- Primary action: **+ New List** opens a `<v-dialog>` with a name `<v-text-field>` and **Create / Cancel**. Use class `oc-cta` on **Create** and **+ New List** (per [ui-style-system.mdc](#)).

- Display owned lists as rows (e.g. `<v-list>` or table): each row shows the **list name** and icon actions:
 - **Edit** icon — opens rename `<v-dialog>` pre-filled with current name; **Save / Cancel**
 - **Delete** icon — opens confirmation `<v-dialog>`
 - (Feature 3 adds an **Items** icon on each row — not in Feature 2)
- Icon-only row actions use `size="small"` and accessible `aria-label`s (**Edit list, Delete list**).
- **Empty state**: "No lists yet. Create your first list." when the user has zero lists.
- **Loading state**: skeleton or progress indicator while lists are fetching.
- **Error state**: `<v-alert type="error">` for API failures.

App chrome

- Introduce `MenuBar` in this feature (not present in Feature 1): signed-in user's name and **Sign out**.
- `MenuBar` is hidden on login and register routes.

Implementation note: one route/view for lists; list CRUD dialogs are child components or inline `<v-dialog>` blocks in `Dashboard.vue` unless the team splits presentational dialogs later.

Key Entities

- **List**: named group belonging to one user; will contain todos (Feature 3).
- **User**: owns many lists (from Feature 1).

Data Model Requirements

Lists table

Field	Type	Rules
id	INTEGER PK	Auto-increment
name	STRING	Required; max 100 chars
userId	INTEGER FK	Required; references users.id; set from req.user.id on create
createdAt	DATE	Sequelize timestamps
updatedAt	DATE	Sequelize timestamps

Associations (in `models/index.js`)

- `User` hasMany `List`
- `List` belongsTo `User`

Acceptance Criteria (Gherkin)

US-2.1 — Create todo lists

Scenario: User creates a new list

- **Given** I am signed in on the dashboard
- **When** I click **+ New List**
- **And** I enter list name `Groceries`
- **And** I confirm the dialog
- **Then** the API returns `201` with a list object containing `id`, `name`, and `userId`

- **And** the returned `userId` matches my authenticated user ID
- **And** `Groceries` appears in the lists view
- **And** the add-list dialog closes

Scenario: User creates a list with an empty name

- **Given** I am signed in on the dashboard
- **When** I open the new list dialog
- **And** I leave the name field empty or whitespace only
- **And** I attempt to confirm
- **Then** inline validation blocks the request
- **And** I see the message **"List name is required."**
- **And** no API request is sent

Scenario: User creates a list with a name that is too long

- **Given** I am signed in on the dashboard
 - **When** I submit a list name longer than 100 characters
 - **Then** the API returns `400` with `{ "message": "List name must be 100 characters or fewer." }`
 - **And** the error is displayed in a `<v-alert type="error">`
-

US-2.2 — View my lists

Scenario: Dashboard loads with existing lists

- **Given** I am signed in
- **And** I own lists `Work` and `Personal`
- **When** I navigate to the dashboard
- **Then** both lists appear in the lists view
- **And** each row shows the list name with edit and delete icon actions

Scenario: User has no lists

- **Given** I am signed in
- **And** I have no lists
- **When** I navigate to the dashboard
- **Then** I see **"No lists yet. Create your first list."**

Scenario: User cannot see another user's lists

- **Given** user B owns list `Secret Project`
 - **And** I am signed in as user A
 - **When** I request `GET /todo/lists`
 - **Then** the response contains only lists owned by user A
 - **And** `Secret Project` is not in the response
 - **And** the lists view does not show `Secret Project`
-

US-2.3 — Manage list rows

Scenario: List rows show edit and delete actions

- **Given** I am signed in
 - **And** I own list `Groceries`
 - **When** I view the dashboard lists view
 - **Then** the `Groceries` row shows an **Edit list** icon action
 - **And** the `Groceries` row shows a **Delete list** icon action
-

US-2.4 — Rename and delete lists

Scenario: User renames a list

- **Given** I am signed in
- **And** I own a list named `Groceries`
- **When** I click the edit icon on the `Groceries` row
- **And** I change the name to `Shopping` in the rename dialog
- **And** I confirm
- **Then** the API returns `200` with the updated list object
- **And** the lists view shows `Shopping` instead of `Groceries`

Scenario: User deletes a list

- **Given** I am signed in
 - **And** I own a list named `Groceries`
 - **When** I click the delete icon on the `Groceries` row
 - **And** I confirm the delete dialog
 - **Then** the API returns `200` or `204`
 - **And** the list is removed from the lists view
-

US-2.5 — Private lists only

Scenario: User attempts to rename another user's list

- **Given** I am signed in as user A
- **And** a list exists that belongs to user B
- **When** I send `PUT /todo/lists/:listId` with user B's list ID and body `{ "name": "Hijacked" }`
- **Then** the API returns `404` with `{ "message": "List with id=<id> not found." }`
- **And** user B's list name is unchanged in the database

Scenario: User attempts to delete another user's list

- **Given** I am signed in as user A
- **And** a list exists that belongs to user B
- **When** I send `DELETE /todo/lists/:listId` with user B's list ID
- **Then** the API returns `404` with `{ "message": "List with id=<id> not found." }`
- **And** user B's list still exists

Scenario: Client cannot assign a list to another user on create

- **Given** I am signed in as user A
- **When** I send `POST /todo/lists` with body `{ "name": "Groceries", "userId": 999 }` where user `999` is a different user
- **Then** the API returns `201` with a list owned by user A
- **And** the saved `userId` is user A's ID, not `999`

Scenario: Unauthenticated user accesses the dashboard

- **Given** I have no session in `localStorage`
- **When** I navigate to the dashboard
- **Then** I am redirected to the login page

Scenario: Unauthenticated API request to lists

- **Given** I have no valid session token
- **When** I request `GET /todo/lists`
- **Then** the API returns `401` with an unauthorized message

Test Coverage Map

Story	Scenario	Test file	Test name
US-2.1	User creates a new list	backend/tests/lists.test.js, frontend/tests/Dashboard.test.js	User creates a new list
US-2.1	User creates a list with an empty name	backend/tests/lists.test.js, frontend/tests/Dashboard.test.js	User creates a list with an empty name
US-2.1	User creates a list with a name that is too long	backend/tests/lists.test.js	User creates a list with a name that is too long
US-2.2	Dashboard loads with existing lists	backend/tests/lists.test.js, frontend/tests/Dashboard.test.js	Dashboard loads with existing lists
US-2.2	User has no lists	frontend/tests/Dashboard.test.js	User has no lists
US-2.2	User cannot see another user's lists	backend/tests/lists.test.js	User cannot see another user's lists
US-2.3	List rows show edit and delete actions	frontend/tests/Dashboard.test.js	List rows show edit and delete actions
US-2.4	User renames a list	backend/tests/lists.test.js, frontend/tests/Dashboard.test.js	User renames a list
US-2.4	User deletes a list	backend/tests/lists.test.js, frontend/tests/Dashboard.test.js	User deletes a list
US-2.5	User attempts to rename another user's list	backend/tests/lists.test.js	User attempts to rename another user's list
US-2.5	User attempts to delete another user's list	backend/tests/lists.test.js	User attempts to delete another user's list
US-2.5	Client cannot assign a list to another user on create	backend/tests/lists.test.js	Client cannot assign a list to another user on create
US-2.5	Unauthenticated API request to lists	backend/tests/lists.test.js	Unauthenticated API request to lists

Agent implementation request

Copy when asking Cursor to implement this feature (@ this file):

```
Implement Feature 2 from @features/feature-2-todo-list-management.md on branch `feature/2-todo-list-management`.
```

```
Follow layer order in @features/framework.md (models → routes → backend tests → frontend → frontend tests).
```

```
Map every Gherkin scenario in the Test Coverage Map; run `npm test` before finishing.
```

If API routes, payloads, schema, or product rules changed per this spec, update @features/reference/api.md, @features/reference/data-model.md, and/or @features/reference/behavior.md in the same PR to match shipped code. Complete Definition of Done and the merge checklist in @features/framework.md. Do not implement behavior not in this spec.

Reference updates for this feature: features/reference/data-model.md , features/reference/api.md , features/reference/behavior.md

Definition of Done

- ☐ Backend and frontend implemented per this spec (**FR-00N** satisfied)
 - ☐ **Success Criteria (SC-00N)** met
 - ☐ All mapped tests pass (`npm test`)
 - ☐ Test Coverage Map complete
 - ☐ features/reference/data-model.md updated (if schema changed)
 - ☐ features/reference/api.md updated (if API changed)
 - ☐ features/reference/behavior.md updated (if product rules changed)
-

Out of Scope

- Todo items (see features/feature-3-todo-list-item-management.md)
 - MenuBar beyond basic sign-out (full nav deferred if not needed)
 - Drag-and-drop list reordering
 - Sharing lists with other users
-

Delivered to Feature 3

The following are intentionally deferred to the next feature spec:

- todos table and associations
- **Items** icon on each list row; list-items dialog (view todos for that list)
- Add / edit / delete todo dialogs and row actions (checkbox, name, edit, delete)
- GET/POST /todo/lists/:listId/todos and PUT/DELETE /todo/todos/:id

feature-3-todo-list-item-management.md

Feature: Todo List Item Management

Feature ID: 3 **Branch pattern:** feature/3-todo-list-item-management **Status:** Ready **Created:** 2026-02-15 **Input:** Signed-in users manage todo items per list via dialogs opened from list rows (items, add, edit, delete) **Depends on:** [Feature 1 — User Authentication](#), [Feature 2 — Todo List Management](#)

User Stories

US-3.1: Add tasks to a list

As a signed-in user

I want to add todo items to a list from its items dialog

So that I can track what needs to be done in that context

Priority: P1

Independent test: Open items dialog for a list, add a todo via add-item dialog; it appears in the items list with `completed: false`

Acceptance scenarios: see ### US-3.1 under Acceptance Criteria

US-3.2: View tasks in a list

As a signed-in user

I want to open a list's items dialog and see all todos for that list

So that I know what work belongs to that group

Priority: P1

Independent test: Open items dialog on different list rows; each dialog shows only that list's todos

Acceptance scenarios: see ### US-3.2 under Acceptance Criteria

US-3.3: Complete tasks

As a signed-in user

I want to mark todos as complete or incomplete

So that I can track my progress

Priority: P1

Independent test: Toggle checkbox; API persists `completed` and UI reflects state

Acceptance scenarios: see ### US-3.3 under Acceptance Criteria

US-3.4: Edit and remove tasks

As a signed-in user

I want to edit or delete individual todos

So that I can keep my lists accurate

Priority: P2

Independent test: Edit title and delete todo via UI; changes persist after refresh

Acceptance scenarios: see ### US-3.4 under Acceptance Criteria

US-3.5: Private items only

As a signed-in user

I want my todo items visible only to me

So that other users cannot read or modify my tasks

Priority: P1

Independent test: Cross-user todo or parent-list access returns `404`

Acceptance scenarios: see ### US-3.5 under Acceptance Criteria

US-3.6: Lists carry their items

As a signed-in user

I want deleting a list to remove its todo items

So that I do not leave orphaned tasks in the database

Priority: P2

Independent test: Delete list with todos; todos are gone from database

Acceptance scenarios: see ### US-3.6 under Acceptance Criteria

Requirements

Functional Requirements

- **FR-001:** All todo endpoints MUST require a valid session (`authenticate` middleware).
- **FR-002:** A todo MUST belong to exactly one list and one user for its entire lifetime.
- **FR-003:** Every todo read, update, and delete MUST scope with `userId: req.user.id` .
- **FR-004:** Before creating a todo, the parent list MUST be owned by `req.user.id` ; otherwise return `404` .
- **FR-005:** On create, `userId` and `listId` MUST come from validated server context — ignore client spoofing of ownership.
- **FR-006:** Todo titles MUST be trimmed before save; empty strings MUST be rejected.
- **FR-007:** New todos MUST default to `completed: false` .
- **FR-008:** Deleting a list MUST delete all todos in that list (cascade).
- **FR-009:** Todos MUST be ordered incomplete first, then by `createdAt` ascending.
- **FR-010:** This feature MUST extend the Feature 2 single-view lists UI: each list row gains an **Items** icon that opens a list-items `<v-dialog>` . Todo add/edit/delete use nested dialogs — no sidebar/main split.

Assumptions

- Features 1–2 MUST be merged to `dev` before implementing this feature (auth, lists, single-view dashboard, `MenuBar` with sign-out).
- Due dates are out of scope (Feature 5).
- No drag-and-drop reorder, search, or sharing.

Edge Cases

- Add todo with items dialog closed → no add UI visible; no API call until user opens items dialog and add-item dialog.
- Empty todo title → client block and/or `400` .
- Title longer than 255 characters → `400` .
- Parent list or todo owned by another user → `404` .
- Unauthenticated todo API → `401` .

Success Criteria

- **SC-001:** Every Gherkin scenario has at least one automated test before merge.
- **SC-002:** User can add, view, complete, edit, and delete todos in an owned list end-to-end.
- **SC-003:** Deleting a list removes its todos; `npm test` passes.

Data Ownership & Isolation

Each user owns their todo items exclusively. Items are private to the user even when nested under a list.

Rule	Requirement
Parent list check	Todo operations require the parent list to belong to <code>req.user.id</code> .
Todo scope	GET, PUT, and DELETE on todos match both <code>id</code> and <code>userId = req.user.id</code> .
Create scope	POST <code>.../todos</code> succeeds only when <code>:listId</code> is owned by the caller; new todo <code>userId</code> is set from <code>req.user.id</code> .
Cross-user access	If a todo or parent list belongs to another user, respond with 404 — never 403.
UI scope	The list-items dialog shows only todos for the list opened from that row, fetched via API for the signed-in user.
Implementation	Use shared helpers (e.g. <code>getAccessibleListOrNull</code> , <code>getAccessibleTodoOrNull</code>) in <code>app/authorization/</code> .

API Requirements

Method	Endpoint	Auth	Purpose
GET	<code>/todo/lists/:listId/todos</code>	Yes	Fetch all todos in a list
POST	<code>/todo/lists/:listId/todos</code>	Yes	Add a todo to a list
PUT	<code>/todo/todos/:id</code>	Yes	Update a todo (title and/or completed)
DELETE	<code>/todo/todos/:id</code>	Yes	Delete a todo owned by the caller

All endpoints enforce **list ownership** and **todo ownership** by the authenticated user. Cross-user access attempts return 404 .

Create todo request body:

```
{ "title": "Buy milk" }
```

Todo success response (200 / 201):

```
{
  "id": 10,
  "listId": 1,
  "title": "Buy milk",
```

```
"completed": false,
"userId": 42,
"createdAt": "2026-07-02T12:05:00.000Z",
"updatedAt": "2026-07-02T12:05:00.000Z"
}
```

Error response: { "message": "Human-readable explanation." } with appropriate HTTP status.

Not found / not owned: 404 (do not use 403).

Screen Requirements

[View: Application Dashboard] — route name `home`

Extends the Feature 2 single-view lists dashboard. List CRUD (add/rename/delete list) is unchanged; this feature adds todo management via dialogs.

List rows (extend Feature 2)

- Each list row adds an **Items** icon (`aria-label : Items` or **View items for <list name>**).
- Clicking **Items** opens a **list-items dialog** for that list.

List-items dialog

- Title shows the list name (e.g. **Groceries — Items**).
- Primary action: **+ Add Item** opens a nested **add-item dialog** with a title `<v-text-field>` and **Add / Cancel**. **+ Add Item** and **Add** use class `oc-cta` .
- Todo rows: **checkbox** (`completed`), **name** (title text), **edit** icon, **delete** icon.
- Edit:** edit icon opens a nested **edit-item dialog** with title field pre-filled; **Save / Cancel**.
- Delete:** delete icon opens a confirmation `<v-dialog>` .
- Completed todos show struck-through or muted title styling.
- Empty state:** "No todos in this list yet." when the list has zero todos.
- Loading state:** skeleton or progress indicator while todos are fetching.
- Error state:** `<v-alert type="error">` for API failures.
- Close:** dialog has **Close** or equivalent to return to the lists view.

List switch behavior

- User opens items dialog on one list row, closes it, then opens items on another row — each dialog load fetches only that list's todos.

Implementation note: list-items, add-item, and edit-item dialogs may be child components; only one list-items dialog need be open at a time.

Key Entities

- Todo:** task item with title and completion state; belongs to one list and one user.
 - List:** parent container for todos (Feature 2); deleting a list removes its todos.
-

Data Model Requirements

`todos` table

Field	Type	Rules
-------	------	-------

id	INTEGER PK	Auto-increment
listId	INTEGER FK	Required; references lists.id; cascade on list delete
title	STRING	Required; max 255 chars
completed	BOOLEAN	Default false
userId	INTEGER FK	Required; references users.id; set from req.user.id on create
createdAt	DATE	Sequelize timestamps
updatedAt	DATE	Sequelize timestamps

Associations (add to `models/index.js`)

- `List` hasMany `Todo` — `onDelete: CASCADE`
- `Todo` belongsTo `List`
- `User` hasMany `Todo`
- `Todo` belongsTo `User`

Acceptance Criteria (Gherkin)

US-3.1 — Add tasks to a list

Scenario: User adds a todo to a list via dialog

- **Given** I am signed in on the dashboard
- **And** I own list `Groceries`
- **When** I click the **Items** icon on the `Groceries` row
- **And** I click **+ Add Item**
- **And** I enter todo title `Buy milk`
- **And** I confirm the add-item dialog
- **Then** the API returns `201` with a todo object where `completed` is `false`
- **And** the returned `userId` matches my authenticated user ID
- **And** the returned `listId` matches `Groceries`
- **And** `Buy milk` appears in the list-items dialog

Scenario: User adds a todo with an empty title

- **Given** I am signed in
- **And** I have opened the items dialog for an owned list
- **When** I open the add-item dialog
- **And** I leave the todo title empty
- **And** I attempt to confirm
- **Then** inline validation blocks the request
- **And** I see the message **"Todo title is required."**
- **And** no API request is sent

Scenario: Add item is only available inside the items dialog

- **Given** I am signed in on the dashboard
 - **And** the list-items dialog is not open
 - **When** I view the lists view
 - **Then** I do not see an add-todo field or **+ Add Item** control on the main lists view
-

US-3.2 — View tasks in a list

Scenario: List items dialog shows empty state

- **Given** I am signed in
- **And** I own an empty list `Personal`
- **When** I open the items dialog for `Personal`
- **And** the todos finish loading
- **Then** I see "No todos in this list yet."

Scenario: User opens items for different lists

- **Given** I am signed in
- **And** list `Work` has todos `Email client` and `Write report`
- **And** list `Personal` has todo `Call mom`
- **When** I open the items dialog for `Personal`
- **Then** I see only `Call mom`
- **When** I close the items dialog
- **And** I open the items dialog for `Work`
- **Then** I see `Email client` and `Write report`

Scenario: User only sees their own todos when opening items

- **Given** I am signed in as user A
 - **And** I own list `Work` with todo `My task`
 - **And** user B owns list `Work` with todo `Their task` (same list name, different owner)
 - **When** I open the items dialog for my `Work` list
 - **Then** I see only `My task`
 - **And** I do not see `Their task`
-

US-3.3 — Complete tasks

Scenario: User marks a todo as complete

- **Given** I am signed in
- **And** I have opened the items dialog for a list containing todo `Buy milk` with `completed: false`
- **When** I check the todo's checkbox
- **Then** the API returns `200` with `completed: true`
- **And** the todo displays as completed (struck-through or muted)

Scenario: User marks a completed todo as incomplete

- **Given** I am signed in
 - **And** I have opened the items dialog for a list containing todo `Buy milk` with `completed: true`
 - **When** I uncheck the todo's checkbox
 - **Then** the API returns `200` with `completed: false`
 - **And** the todo displays as active again
-

US-3.4 — Edit and remove tasks

Scenario: User edits a todo title

- **Given** I am signed in
- **And** I have opened the items dialog for a list containing todo `Buy milk`
- **When** I click the edit icon on `Buy milk`
- **And** I change the title to `Buy oat milk` in the edit dialog

- **And** I confirm
- **Then** the API returns `200` with the updated title
- **And** the list-items dialog shows `Buy oat milk`

Scenario: User deletes a todo

- **Given** I am signed in
 - **And** I have opened the items dialog for a list containing todo `Buy milk`
 - **When** I click the delete icon on `Buy milk`
 - **And** I confirm
 - **Then** the API returns `200` or `204`
 - **And** the todo is removed from the list-items dialog
-

US-3.5 — Private items only

Scenario: User cannot read todos in another user's list

- **Given** I am signed in as user A
- **And** user B owns list `Secret` with todo `Hidden task`
- **When** I request `GET /todo/lists/:listId/todos` with user B's list ID
- **Then** the API returns `404` with `{ "message": "List with id=<id> not found." }`
- **And** `Hidden task` is not returned to user A

Scenario: User attempts to add a todo to another user's list

- **Given** I am signed in as user A
- **And** a list exists that belongs to user B
- **When** I send `POST /todo/lists/:listId/todos` with user B's list ID and body `{ "title": "Intruder task" }`
- **Then** the API returns `404` with `{ "message": "List with id=<id> not found." }`
- **And** no todo is created in user B's list

Scenario: User attempts to rename another user's todo

- **Given** I am signed in as user A
- **And** a todo exists that belongs to user B
- **When** I send `PUT /todo/todos/:id` with body `{ "title": "Hijacked" }`
- **Then** the API returns `404` with `{ "message": "Todo with id=<id> not found." }`
- **And** user B's todo title is unchanged in the database

Scenario: User attempts to delete another user's todo

- **Given** I am signed in as user A
- **And** a todo exists that belongs to user B
- **When** I send `DELETE /todo/todos/:id`
- **Then** the API returns `404` with `{ "message": "Todo with id=<id> not found." }`
- **And** user B's todo still exists

Scenario: Client cannot assign a todo to another user on create

- **Given** I am signed in as user A
- **And** I own list `Groceries`
- **When** I send `POST /todo/lists/:listId/todos` with body `{ "title": "Buy milk", "userId": 999 }` where user `999` is a different user
- **Then** the API returns `201` with a todo owned by user A
- **And** the saved `userId` is user A's ID, not `999`

Scenario: Unauthenticated API request for todos

- **Given** I have no valid session token

- **When** I request `GET /todo/lists/1/todos`
 - **Then** the API returns `401` with an unauthorized message
-

US-3.6 — Lists carry their items

Scenario: Deleting a list removes its todos

- **Given** I am signed in
 - **And** I own list `Groceries` with todos `Buy milk` and `Buy eggs`
 - **When** I delete list `Groceries` and confirm
 - **Then** both todos are removed from the database
 - **And** they no longer appear if the list ID were still queried
-

Test Coverage Map

Story	Scenario	Test file	Test name
US-3.1	User adds a todo to a list via dialog	backend/tests/todos.test.js, frontend/tests/Dashboard.test.js	User adds a todo to a list via dialog
US-3.1	User adds a todo with an empty title	backend/tests/todos.test.js, frontend/tests/Dashboard.test.js	User adds a todo with an empty title
US-3.1	Add item is only available inside the items dialog	frontend/tests/Dashboard.test.js	Add item is only available inside the items dialog
US-3.2	List items dialog shows empty state	frontend/tests/Dashboard.test.js	List items dialog shows empty state
US-3.2	User opens items for different lists	frontend/tests/Dashboard.test.js	User opens items for different lists
US-3.2	User only sees their own todos when opening items	backend/tests/todos.test.js	User only sees their own todos when opening items
US-3.3	User marks a todo as complete	backend/tests/todos.test.js, frontend/tests/Dashboard.test.js	User marks a todo as complete
US-3.3	User marks a completed todo as incomplete	backend/tests/todos.test.js, frontend/tests/Dashboard.test.js	User marks a completed todo as incomplete
US-3.4	User edits a todo title	backend/tests/todos.test.js, frontend/tests/Dashboard.test.js	User edits a todo title
US-3.4	User deletes a todo	backend/tests/todos.test.js, frontend/tests/Dashboard.test.js	User deletes a todo
US-3.5	User cannot read todos in another user's list	backend/tests/todos.test.js	User cannot read todos in another user's list
US-3.5	User attempts to add a todo to another user's list	backend/tests/todos.test.js	User attempts to add a todo to another user's list
US-3.5	User attempts to rename another user's todo	backend/tests/todos.test.js	User attempts to rename another user's todo

US-3.5	User attempts to delete another user's todo	backend/tests/todos.test.js	User attempts to delete another user's todo
US-3.5	Client cannot assign a todo to another user on create	backend/tests/todos.test.js	Client cannot assign a todo to another user on create
US-3.5	Unauthenticated API request for todos	backend/tests/todos.test.js	Unauthenticated API request for todos
US-3.6	Deleting a list removes its todos	backend/tests/todos.test.js	Deleting a list removes its todos

Agent implementation request

Copy when asking Cursor to implement this feature (@ this file):

Implement Feature 3 from @features/feature-3-todo-list-item-management.md on branch `feature/3-todo-list-item-management`.

Follow layer order in @features/framework.md (models → routes → backend tests → frontend → frontend tests).

Map every Gherkin scenario in the Test Coverage Map; run `npm test` before finishing.

If API routes, payloads, schema, or product rules changed per this spec, update @features/reference/api.md, @features/reference/data-model.md, and/or @features/reference/behavior.md in the same PR to match shipped code.

Complete Definition of Done and the merge checklist in @features/framework.md.

Do not implement behavior not in this spec.

Reference updates for this feature: features/reference/data-model.md , features/reference/api.md , features/reference/behavior.md

Definition of Done

- ☐ Backend and frontend implemented per this spec (**FR-00N** satisfied)
- ☐ **Success Criteria (SC-00N)** met
- ☐ All mapped tests pass (`npm test`)
- ☐ Test Coverage Map complete
- ☐ features/reference/data-model.md updated (if schema changed)
- ☐ features/reference/api.md updated (if API changed)
- ☐ features/reference/behavior.md updated (if product rules changed)

Out of Scope

- New list CRUD features (owned by Feature 2)
- Drag-and-drop reordering of todos
- Due dates → [feature-5-todo-due-date.md](#) (Feature 5)
- Priorities, labels, or notes on todos
- Sharing lists or todos with other users
- Search or filter across todos

- Bulk complete / bulk delete
- Archive completed todos

feature-4-user-profile-management.md

Feature: User Profile Management

Feature ID: 4 **Branch pattern:** feature/4-user-profile-management **Status:** Ready **Created:** 2026-03-01 **Input:**

Signed-in users view and edit their profile from a menu-bar dropdown; logout moves to profile menu **Depends on:** [Feature 1 — User Authentication](#), [Feature 2 — Todo List Management](#), [Feature 3 — Todo List Item Management](#)

User Stories

US-4.1: View profile from the menu bar

As a signed-in user

I want to open a profile dropdown from a user icon on the menu bar

So that I can see my name, username, and email at a glance

Priority: P1

Independent test: Open profile dropdown; name, username, and email are visible

Acceptance scenarios: see ### US-4.1 under Acceptance Criteria

US-4.2: Edit profile

As a signed-in user

I want to edit my profile

So that I can change my name, username, email, and password

Priority: P1

Independent test: Save valid profile changes; API and `localStorage` reflect updates

Acceptance scenarios: see ### US-4.2 under Acceptance Criteria

US-4.3: Log out from profile

As a signed-in user

I want to see a **Log out** action in the profile dropdown

So that I can end my session

Priority: P2

Independent test: Log out from dropdown clears session and redirects to login

Acceptance scenarios: see ### US-4.3 under Acceptance Criteria

US-4.4: Single logout entry point

As a signed-in user

I want the menu bar **Sign out** button removed

So that logout lives in one consistent place (the profile dropdown)

Priority: P2

Independent test: Menu bar has no standalone **Sign out** button

Acceptance scenarios: see ### US-4.4 under Acceptance Criteria

Requirements

Functional Requirements

- **FR-001:** All profile endpoints MUST require a valid session (`authenticate` middleware).
- **FR-002:** A user MAY read and update only their own profile row (`id` MUST match `req.user.id`).
- **FR-003:** Cross-user profile access MUST return `404` — never `403` .
- **FR-004:** Profile fields MUST be trimmed before save; empty required strings MUST be rejected.
- **FR-005:** Password updates MUST be optional on `PUT` ; when provided, enforce minimum 8 characters and bcrypt hash before save.
- **FR-006:** Username MUST be normalized on save: `trim().toLowerCase()` .
- **FR-007:** Responses MUST never include the password hash.
- **FR-008:** After successful profile update, the frontend MUST refresh `localStorage` key `user` and dispatch `user-logged-in` so `MenuBar` reflects the new display name.
- **FR-009:** Edit Profile MUST use shared `emailRules` from `frontend/src/config/validation.js` (same as registration).
- **FR-010:** Dashboard list and todo behavior MUST remain unchanged (Features 2–3).

Assumptions

- Features 1–3 MUST be merged to `dev` before implementing this feature.
- No new database tables — profile uses existing `users` from Feature 1.
- Role is read-only; no admin user management.

Edge Cases

- Fetch or update another user's profile → `404` .
- Duplicate username or email on update → `400` .
- Optional password omitted on update → existing password unchanged.
- Invalid email format or short password → client block and/or `400` .
- Unauthenticated profile API → `401` .

Success Criteria

- **SC-001:** Every Gherkin scenario has at least one automated test before merge.
- **SC-002:** User can view profile, edit fields, and log out from the profile dropdown only.
- **SC-003:** `npm test` passes for `users.test.js` and `MenuBar.test.js` .

Data Ownership & Isolation

Each user manages their own profile exclusively.

Rule	Requirement
Read scope	<code>GET /todo/users/:id</code> succeeds only when <code>:id = req.user.id</code> .
Write scope	<code>PUT /todo/users/:id</code> applies only when <code>:id = req.user.id</code> .
Cross-user access	If <code>:id</code> belongs to another user, respond with <code>404</code> — do not confirm the user exists.

Implementation

Use a shared helper (e.g. `getAccessibleUserOrNull(req, userId)`) in `app/authorization/` — do not duplicate scope logic in controllers.

API Requirements

Method	Endpoint	Auth	Purpose
GET	<code>/todo/users/:id</code>	Yes	Fetch the authenticated user's profile
PUT	<code>/todo/users/:id</code>	Yes	Update the authenticated user's profile

All endpoints enforce **self-access only**. Cross-user access attempts return `404`.

Update profile request body:

```
{
  "fName": "Jane",
  "lName": "Doe",
  "email": "jane@example.com",
  "username": "jdoe",
  "password": "newpassword123"
}
```

`password` is optional. Omit it to leave the current password unchanged.

Profile success response (`200`):

```
{
  "id": 42,
  "fName": "Jane",
  "lName": "Doe",
  "email": "jane@example.com",
  "username": "jdoe",
  "role": "worker",
  "createdAt": "2026-07-02T12:00:00.000Z",
  "updatedAt": "2026-07-02T12:05:00.000Z"
}
```

Error response: `{ "message": "Human-readable explanation." }` with appropriate HTTP status.

Not found / not owned: `404` (do not use `403`).

Screen Requirements

[Component: MenuBar] — all authenticated routes

Extends the Feature 2 `MenuBar`. Dashboard lists view is unchanged.

Menu bar changes (this feature)

- Replace the inline display name + **Sign out** button with a **user icon** (`mdi-account-circle` or similar).
- Clicking the user icon opens a `<v-menu>` profile dropdown.

- **Remove** the standalone **Sign out** button from the app bar.

Profile dropdown (`<v-menu>`)

- Read-only display of **full name** (`fName` + `lName`), **username**, and **email**.
- Use `<v-list-item>` with the full name as the title and username/email as subtitle lines.
- **Edit Profile** button opens the edit dialog. Use class `oc-cta` — this is the reference label size for peer CTAs (+ **New List**, **Add**).
- **Log out** list item or button — reuses existing logout flow (`authServices.logoutUser()`).

Frontend services

- Add `userServices.js` with `getUser(userId)` and `updateUser(userId, payload)`.

Edit Profile dialog (`<v-dialog>`)

- `<v-text-field>` for first name, last name, email, username.
- Optional `<v-text-field type="password">` for new password and confirm password.
- Pre-fill all fields except passwords from the current session / `GET /todo/users/:id`.
- **Save / Cancel** actions.
- Client-side validation mirrors `Register.vue` rules (required fields, email format via shared `emailRules`, password length, password match).
- **Loading state:** `:loading` on **Save** while the API request is in flight.
- **Error state:** `<v-alert type="error">` for API failures.

Logout

- **Log out** in the profile dropdown replaces menu-bar **Sign out** (same API and redirect behavior as Feature 1).

Key Entities

- **User:** same entity as Feature 1; profile fields editable via API (no new tables).

Data Model Requirements

No new tables. This feature uses the existing `users` table from Feature 1.

Field	Notes for this feature
<code>fName</code> , <code>lName</code> , <code>email</code> , <code>username</code>	Editable via <code>PUT /todo/users/:id</code>
<code>password</code>	Optional on update; hashed when provided
<code>role</code>	Read-only in API responses; not editable in this feature

Acceptance Criteria (Gherkin)

US-4.1 — View profile from the menu bar

Scenario: User opens the profile dropdown from the menu bar

- **Given** I am signed in on the dashboard
- **When** I click the user icon on the menu bar
- **Then** the profile dropdown is displayed
- **And** the dropdown shows my full name (`fName` + `lName`)

- **And** the dropdown shows my username
 - **And** the dropdown shows my email
 - **And** an **Edit Profile** button is displayed
 - **And** a **Log out** action is displayed
-

US-4.2 — Edit profile

Scenario: User opens the edit profile dialog

- **Given** I am signed in
- **And** the profile dropdown is displayed
- **When** I click **Edit Profile**
- **Then** the Edit Profile dialog is displayed
- **And** fields are pre-filled with my current first name, last name, email, and username

Scenario: User cancels the edit profile dialog

- **Given** I am signed in
- **And** the Edit Profile dialog is displayed
- **When** I change one or more fields
- **And** I click **Cancel**
- **Then** the Edit Profile dialog closes
- **And** no profile update API request is sent
- **And** my stored profile data is unchanged

Scenario: User saves profile changes

- **Given** I am signed in
- **And** the Edit Profile dialog is displayed
- **When** I update my first name, last name, email, or username with valid values
- **And** I click **Save**
- **Then** the API returns `200` with the updated user object (no password hash)
- **And** the Edit Profile dialog closes
- **And** `localStorage` key `user` is updated
- **And** reopening the profile dropdown shows my updated full name, username, and email

Scenario: User saves profile with invalid email format

- **Given** I am signed in
- **And** the Edit Profile dialog is displayed
- **When** I enter a value that is not a valid email address (e.g. `notanemail`)
- **And** I click **Save**
- **Then** inline validation blocks the request
- **And** I see the message **"Enter a valid email address."**
- **And** no profile update API request is sent

Scenario: User saves profile with mismatched passwords

- **Given** I am signed in
- **And** the Edit Profile dialog is displayed
- **When** I enter a new password and a non-matching confirmation
- **And** I click **Save**
- **Then** inline validation blocks the request
- **And** I see the message **"Passwords do not match."**
- **And** no profile update API request is sent

Scenario: User saves profile with a password that is too short

- **Given** I am signed in
- **And** the Edit Profile dialog is displayed
- **When** I enter a new password shorter than 8 characters with a matching confirmation
- **And** I click **Save**
- **Then** inline validation blocks the request
- **And** I see the message **"Password must be at least 8 characters."**
- **And** no profile update API request is sent

Scenario: Profile update API returns an error

- **Given** I am signed in
- **And** the Edit Profile dialog is displayed
- **When** I click **Save**
- **And** the API returns `400` with `{ "message": "..."}`
- **Then** the error is displayed in a `<v-alert type="error">`
- **And** the Edit Profile dialog remains open

Scenario: User fetches their own profile

- **Given** I am signed in as user A
- **When** I request `GET /todo/users/:id` with my user ID
- **Then** the API returns `200` with my profile fields
- **And** the response does not include a password hash

Scenario: User attempts to fetch another user's profile

- **Given** I am signed in as user A
- **And** user B exists
- **When** I request `GET /todo/users/:id` with user B's ID
- **Then** the API returns `404` with `{ "message": "User with id=<id> not found." }`

Scenario: User attempts to update another user's profile

- **Given** I am signed in as user A
- **And** user B exists
- **When** I send `PUT /todo/users/:id` with user B's ID
- **Then** the API returns `404` with `{ "message": "User with id=<id> not found." }`
- **And** user B's profile is unchanged in the database

Scenario: Unauthenticated profile API request

- **Given** I have no valid session token
- **When** I request `GET /todo/users/1`
- **Then** the API returns `401` with an unauthorized message

Scenario: Profile update rejects a password that is too short

- **Given** I am signed in as user A
- **When** I send `PUT /todo/users/:id` with body `{ "password": "short" }`
- **Then** the API returns `400` with `{ "message": "Password must be at least 8 characters." }`

Scenario: Profile update rejects missing required fields

- **Given** I am signed in as user A
- **When** I send `PUT /todo/users/:id` with a body that omits a required field (e.g. first name)
- **Then** the API returns `400` with `{ "message": "First name is required." }`
- **And** my stored profile is unchanged

Scenario: Profile update rejects a duplicate username

- **Given** I am signed in as user A
- **And** user B exists with username `userb`
- **When** I send `PUT /todo/users/:id` with body `{ "username": "userb" }` (and other valid fields)
- **Then** the API returns `400` with `{ "message": "Username is already taken." }`
- **And** user B's username remains `userb`

Scenario: Profile update rejects a duplicate email

- **Given** I am signed in as user A
- **And** user B exists with email `b@example.com`
- **When** I send `PUT /todo/users/:id` with body `{ "email": "b@example.com" }` (and other valid fields)
- **Then** the API returns `400` with `{ "message": "Email is already registered." }`
- **And** user B's email remains `b@example.com`

Scenario: Unauthenticated profile update API request

- **Given** I have no valid session token
- **When** I send `PUT /todo/users/1` with a valid profile body
- **Then** the API returns `401` with an unauthorized message

US-4.3 — Log out from profile

Scenario: User logs out from the profile dropdown

- **Given** I am signed in on the dashboard
- **And** the profile dropdown is open
- **When** I click **Log out**
- **Then** the API invalidates my session token on the server
- **And** `localStorage` key `user` is removed
- **And** I am redirected to the login page

US-4.4 — Single logout entry point

Scenario: Menu bar does not show Sign out

- **Given** I am signed in on the dashboard
- **When** I view the menu bar
- **Then** I do not see a **Sign out** button on the menu bar

Test Coverage Map

Each scenario above must map to at least one automated test.

Story	Scenario	Test file	Test name
US-4.1	User opens the profile dropdown from the menu bar	<code>frontend/tests/MenuBar.test.js</code>	User opens the profile dropdown from the menu bar
US-4.2	User opens the edit profile dialog	<code>frontend/tests/MenuBar.test.js</code>	User opens the edit profile dialog
US-4.2	User cancels the edit profile dialog	<code>frontend/tests/MenuBar.test.js</code>	User cancels the edit profile dialog

US-4.2	User saves profile changes	backend/tests/users.test.js, frontend/tests/MenuBar.test.js	User saves profile changes
US-4.2	User saves profile with invalid email format	frontend/tests/MenuBar.test.js	User saves profile with invalid email format
US-4.2	User saves profile with mismatched passwords	frontend/tests/MenuBar.test.js	User saves profile with mismatched passwords
US-4.2	User saves profile with a password that is too short	frontend/tests/MenuBar.test.js	User saves profile with a password that is too short
US-4.2	Profile update API returns an error	frontend/tests/MenuBar.test.js	Profile update API returns an error
US-4.2	User fetches their own profile	backend/tests/users.test.js	User fetches their own profile
US-4.2	User attempts to fetch another user's profile	backend/tests/users.test.js	User attempts to fetch another user's profile
US-4.2	User attempts to update another user's profile	backend/tests/users.test.js	User attempts to update another user's profile
US-4.2	Unauthenticated profile API request	backend/tests/users.test.js	Unauthenticated profile API request
US-4.2	Profile update rejects a password that is too short	backend/tests/users.test.js	Profile update rejects a password that is too short
US-4.2	Profile update rejects missing required fields	backend/tests/users.test.js	Profile update rejects missing required fields
US-4.2	Profile update rejects a duplicate username	backend/tests/users.test.js	Profile update rejects a duplicate username
US-4.2	Profile update rejects a duplicate email	backend/tests/users.test.js	Profile update rejects a duplicate email
US-4.2	Unauthenticated profile update API request	backend/tests/users.test.js	Unauthenticated profile update API request
US-4.3	User logs out from the profile dropdown	frontend/tests/MenuBar.test.js	User logs out from the profile dropdown
US-4.4	Menu bar does not show Sign out	frontend/tests/MenuBar.test.js	Menu bar does not show Sign out

Agent implementation request

Copy when asking Cursor to implement this feature (@ this file):

```
Implement Feature 4 from @features/feature-4-user-profile-management.md on branch `feature/4-user-profile-management`.
```


Follow layer order in @features/framework.md (models → routes → backend tests → frontend → frontend tests).

Map every Gherkin scenario in the Test Coverage Map; run `npm test` before finishing.

If API routes, payloads, schema, or product rules changed per this spec, update @features/reference/api.md, @features/reference/data-model.md, and/or @features/reference/behavior.md in the same PR to match shipped code.

Complete Definition of Done and the merge checklist in @features/framework.md.

Do not implement behavior not in this spec.

Reference updates for this feature: features/reference/data-model.md , features/reference/api.md , features/reference/behavior.md

Definition of Done

- ☐ Backend and frontend implemented per this spec (**FR-00N** satisfied)
 - ☐ **Success Criteria (SC-00N)** met
 - ☐ All mapped tests pass (`npm test`)
 - ☐ Test Coverage Map complete
 - ☐ features/reference/data-model.md updated (if schema changed)
 - ☐ features/reference/api.md updated (if API changed)
 - ☐ features/reference/behavior.md updated (if product rules changed)
-

Out of Scope

- Admin user management or role changes
- Avatar or profile photo upload
- Email verification workflow
- Changes to list or todo CRUD (Features 2–3)
- Password reset / forgot-password flow

feature-5-todo-due-date.md

Feature: Todo Due Date

Feature ID: 5 **Branch pattern:** feature/5-todo-due-date **Status:** Ready **Created:** 2026-03-15 **Input:** Optional calendar due dates on todos with display and overdue highlighting **Depends on:** [Feature 1 — User Authentication](#), [Feature 2 — Todo List Management](#), [Feature 3 — Todo List Item Management](#) **Related:** features/reference/data-model.md , features/reference/api.md , features/reference/behavior.md (update in same PR when implementing)

User Stories

US-5.1: Set a due date when creating a todo

As a signed-in user

I want to optionally set a due date when I add a todo

So that I can plan when work should be finished

Priority: P1

Independent test: Create todo with dueDate ; API returns date and row displays it

Acceptance scenarios: see ### US-5.1 under Acceptance Criteria

US-5.2: View due dates on todos

As a signed-in user

I want to see each todo's due date in the list

So that I know what is due and when

Priority: P1

Independent test: Todos with dueDate show formatted date in list rows

Acceptance scenarios: covered by US-5.1 and US-5.3 scenarios (display in list)

US-5.3: Edit or clear a due date

As a signed-in user

I want to change or remove a due date when editing a todo

So that I can keep deadlines accurate

Priority: P1

Independent test: Edit dialog sets or clears dueDate ; API and UI stay in sync

Acceptance scenarios: see ### US-5.3 under Acceptance Criteria

US-5.4: Spot overdue todos

As a signed-in user

I want incomplete todos past their due date to stand out visually

So that I can prioritize overdue work

Priority: P2

Independent test: Incomplete todo with past dueDate uses overdue styling; completed does not

Acceptance scenarios: see ### US-5.4 under Acceptance Criteria

Requirements

Functional Requirements

- **FR-001:** All behavior MUST build on Feature 3 todo CRUD (list-items and add/edit-item dialogs); list and ownership rules are unchanged.
- **FR-002:** `dueDate` MUST be optional on create and update; `null` means no due date.
- **FR-003:** Dates MUST be calendar-only: API `YYYY-MM-DD` ; database `DATEONLY` .
- **FR-004:** Invalid date strings MUST return `400` with `{ "message": "..."}` .
- **FR-005:** Sending `dueDate: null` on `PUT` MUST clear the due date.
- **FR-006:** Omitting `dueDate` on `PUT` MUST leave the existing value unchanged.
- **FR-007:** Todo sort order MUST remain unchanged from Feature 3 (incomplete first, then `createdAt` ascending).
- **FR-008:** Incomplete todos MUST be styled overdue when `dueDate` is before today in the browser's local calendar (frontend only; API returns stored date).
- **FR-009:** Reference docs MUST be updated in the same PR when implementing (see **Agent implementation request**).

Assumptions

- Features 1–3 MUST be merged to `dev` before implementing this feature (todo CRUD complete).
- No timezone or time-of-day — date-only in local browser calendar.
- No sorting, filtering, reminders, or calendar views.

Edge Cases

- Invalid `dueDate` on create or update → `400` (`"Due date must be a valid date in YYYY-MM-DD format."`).
- Cross-user todo `dueDate` change → `404` .
- Completed todo with past due date → no overdue styling.
- Create without `dueDate` → `null` in API and no date on row.

Success Criteria

- **SC-001:** Every Gherkin scenario has at least one automated test before merge.
- **SC-002:** User can set, view, edit, and clear due dates on owned todos.
- **SC-003:** Overdue styling applies only to incomplete past-due todos; `npm test` passes.

Data Ownership & Isolation

Due date changes follow the same scope rules as Feature 3 todos.

Rule	Requirement
Read scope	<code>dueDate</code> is returned only on todos the caller already owns via list/todo scoping.
Write scope	<code>dueDate</code> may be set or cleared only on todos owned by <code>req.user.id</code> .
Cross-user access	Unchanged — <code>404</code> for another user's list or todo.

API Requirements

Extends Feature 3 todo endpoints. Auth and ownership behavior are unchanged.

Method	Endpoint	Change
POST	/todo/lists/:listId/todos	Accept optional dueDate in body
PUT	/todo/todos/:id	Accept optional dueDate (date string or null)
GET	/todo/lists/:listId/todos	Response includes dueDate on each todo

Create todo request body:

```
{
  "title": "Buy milk",
  "dueDate": "2026-07-15"
}
```

`dueDate` is optional. Omit it or send `null` for no due date.

Update todo request body (any combination):

```
{
  "title": "Buy oat milk",
  "completed": false,
  "dueDate": "2026-07-20"
}
```

Clear due date:

```
{ "dueDate": null }
```

Todo success response (200 / 201):

```
{
  "id": 10,
  "listId": 1,
  "title": "Buy milk",
  "completed": false,
  "dueDate": "2026-07-15",
  "userId": 42,
  "createdAt": "2026-07-02T12:05:00.000Z",
  "updatedAt": "2026-07-02T12:05:00.000Z"
}
```

`dueDate` is `null` when not set.

Validation errors: 400 with `{ "message": "..."}` for invalid `dueDate` format.

Error response: unchanged from Feature 3.
Not found / not owned: 404 (do not use 403).

Screen Requirements

[View: Application Dashboard] — route name home

Extends Feature 3 list-items, add-item, and edit-item dialogs only.

Add-item dialog

- Optional `<v-text-field type="date">` (or equivalent) beside the title field for due date.
- Leaving the date empty creates a todo with no due date.

Todo row (in list-items dialog)

- Show due date when set (formatted for readability, e.g. Jul 15, 2026 or locale-appropriate).
- When `completed` is `false` and `dueDate` is before today (local date), apply overdue styling (e.g. error color on the date text).
- Completed todos do not use overdue styling even if the date is in the past.

Edit todo dialog

- Add optional date field pre-filled with the current `dueDate` (empty when `null`).
- User can clear the date and **Save** to remove the due date.
- **Save** / **Cancel** behavior unchanged otherwise.

Validation

- Client-side: reject invalid date input before API call where the control allows it.
 - API errors shown via existing `<v-alert type="error">` .
-

Key Entities

- **Todo**: gains optional **dueDate** (calendar date); still belongs to one list and one user.
-

Data Model Requirements

todos table (add column)

Field	Type	Rules
dueDate	DATEONLY	Nullable; optional on create/update

Existing Feature 3 columns are unchanged. Existing rows default to `dueDate: null` .

Acceptance Criteria (Gherkin)

US-5.1 — Set a due date when creating a todo

Scenario: User adds a todo with a due date

- **Given** I am signed in on the dashboard
- **And** I have opened the items dialog for an owned list

- **When** I click **+ Add Item**
- **And** I enter todo title `Buy milk`
- **And** I set due date `2026-07-15`
- **And** I confirm the add-item dialog
- **Then** the API returns `201` with `dueDate 2026-07-15`
- **And** the todo row in the list-items dialog shows the due date

Scenario: User adds a todo without a due date

- **Given** I am signed in
- **And** I have opened the items dialog for an owned list
- **When** I open the add-item dialog
- **And** I enter a title and leave the due date empty
- **And** I confirm
- **Then** the API returns `201` with `dueDate null`
- **And** no due date is shown on the row

Scenario: API rejects an invalid due date on create

- **Given** I am signed in as user A
- **And** I own a list
- **When** I send `POST /todo/lists/:listId/todos` with body `{ "title": "Task", "dueDate": "not-a-date" }`
- **Then** the API returns `400` with `{ "message": "..."}`
- **And** no todo is created

US-5.3 — Edit or clear a due date

Scenario: User sets a due date when editing a todo

- **Given** I am signed in
- **And** I have todo `Buy milk` with no due date
- **When** I open the edit dialog
- **And** I set due date `2026-07-20`
- **And** I click **Save**
- **Then** the API returns `200` with `dueDate 2026-07-20`
- **And** the row shows the new due date

Scenario: User clears a due date when editing a todo

- **Given** I am signed in
- **And** I have todo `Buy milk` with due date `2026-07-20`
- **When** I open the edit dialog
- **And** I clear the due date field
- **And** I click **Save**
- **Then** the API returns `200` with `dueDate null`
- **And** the row no longer shows a due date

Scenario: API rejects an invalid due date on update

- **Given** I am signed in as user A
- **And** I own todo `Buy milk`
- **When** I send `PUT /todo/todos/:id` with body `{ "dueDate": "2026-99-99" }`
- **Then** the API returns `400` with `{ "message": "..."}`
- **And** the stored `dueDate` is unchanged

Scenario: User cannot set due date on another user's todo

- **Given** I am signed in as user A
- **And** a todo exists that belongs to user B
- **When** I send `PUT /todo/todos/:id` with body `{ "dueDate": "2026-07-15" }`
- **Then** the API returns `404` with `{ "message": "Todo with id=<id> not found." }`
- **And** user B's todo is unchanged

US-5.4 — Spot overdue todos

Scenario: Incomplete todo past due date is styled as overdue

- **Given** I am signed in
- **And** I have opened the items dialog for a list containing an incomplete todo with `dueDate` yesterday
- **When** the todos are displayed
- **Then** the due date is displayed with overdue styling

Scenario: Completed todo past due date is not styled as overdue

- **Given** I am signed in
- **And** I have opened the items dialog for a list containing a completed todo with `dueDate` yesterday
- **When** the todos are displayed
- **Then** the due date does not use overdue styling

Test Coverage Map

Each scenario above must map to at least one automated test.

Story	Scenario	Test file	Test name
US-5.1	User adds a todo with a due date	backend/tests/todos.test.js, frontend/tests/Dashboard.test.js	User adds a todo with a due date
US-5.1	User adds a todo without a due date	backend/tests/todos.test.js	User adds a todo without a due date
US-5.1	API rejects an invalid due date on create	backend/tests/todos.test.js	API rejects an invalid due date on create
US-5.3	User sets a due date when editing a todo	backend/tests/todos.test.js, frontend/tests/Dashboard.test.js	User sets a due date when editing a todo
US-5.3	User clears a due date when editing a todo	backend/tests/todos.test.js, frontend/tests/Dashboard.test.js	User clears a due date when editing a todo
US-5.3	API rejects an invalid due date on update	backend/tests/todos.test.js	API rejects an invalid due date on update
US-5.3	User cannot set due date on another user's todo	backend/tests/todos.test.js	User cannot set due date on another user's todo
US-5.4	Incomplete todo past due date is styled as overdue	frontend/tests/Dashboard.test.js	Incomplete todo past due date is styled as overdue

US-5.4	Completed todo past due date is not styled as overdue	frontend/tests/Dashboard.test.js	Completed todo past due date is not styled as overdue
--------	---	----------------------------------	---

Agent implementation request

Copy when asking Cursor to implement this feature (@ this file):

```
Implement Feature 5 from @features/feature-5-todo-due-date.md on branch `feature/5-todo-due-date`.
```

```
Follow layer order in @features/framework.md (models → routes → backend tests → frontend → frontend tests).
```

```
Map every Gherkin scenario in the Test Coverage Map; run `npm test` before finishing.
```

```
If API routes, payloads, schema, or product rules changed per this spec, update @features/reference/api.md, @features/reference/data-model.md, and/or @features/reference/behavior.md in the same PR to match shipped code.
```

```
Complete Definition of Done and the merge checklist in @features/framework.md.
```

```
Do not implement behavior not in this spec.
```

Reference updates for this feature: features/reference/data-model.md , features/reference/api.md , features/reference/behavior.md

Definition of Done

- ☐ Backend and frontend implemented per this spec (**FR-00N** satisfied)
- ☐ **Success Criteria (SC-00N)** met
- ☐ All mapped tests pass (npm test)
- ☐ Test Coverage Map complete
- ☐ features/reference/data-model.md updated
- ☐ features/reference/api.md updated
- ☐ features/reference/behavior.md updated (if product rules changed)

Out of Scope

- Sorting or filtering todos by due date
- Due date on quick-add without opening edit dialog (optional field on add row is in scope; separate due-date-only modal is not)
- Reminders, notifications, or email alerts
- Recurring todos
- Time-of-day or timezone handling (date-only)
- Calendar or agenda views
- Changes to lists, profile, or auth (Features 2, 4)

Part 6: Reference (current integrated state)

README.md

Reference Specifications

Living snapshot of the integrated product on `dev` after merged features.

These files answer: *"What does the app look like right now?"*

They do **not** authorize new scope — implement only from `features/feature-*.md` .

Student guide: [writing-living-reference.md](#) (PDF) — when/how to update api, data-model, and behavior.

Maintenance

When	Action
Feature merges to dev	Required DoD: update the matching reference file(s) in the same PR — <code>data-model.md</code> / <code>api.md</code> if schema or routes/payloads changed; behavior.md if product rules changed (sort, ownership, validation, UI rules). See Merge checklist + Agility sync
Feature in progress	Each <code>feature-N-*.md</code> includes an Agent implementation request block — paste or @ the spec so Cursor updates reference in the same implementation PR
New feature in progress	Feature spec owns the delta ; reference updates with implementation, not as optional post-merge cleanup
Drift suspected	Compare reference to code and feature specs; fix reference or code

Files

File	Contents
data-model.md	Current database tables, columns, associations
api.md	Current REST API
behavior.md	Current product rules (ownership, sort, validation, UI rules) — not API/schema shapes

Feature provenance

Area	Introduced
Auth, sessions	Feature 1
Lists CRUD + Dashboard lists view + MenuBar (sign-out era)	Feature 2
Todos CRUD + items dialogs	Feature 3
Profile API + MenuBar profile dropdown / Log out	Feature 4

Todo dueDate + overdue styling

Feature 5

api.md

API Reference

Base path: `/todo/`

Status: Integrated API through **Feature 5** (authentication, lists, todos with optional due dates, user profile).

Authority for new work: feature specs in `features/` — update this file in the same PR when routes or payloads change.

Auth: Send `Authorization: Bearer <token>` on protected routes.

Errors: `{ "message": "Human-readable explanation." }` unless noted.

Feature provenance

Area	Feature
Register, login, logout	1
List CRUD (GET/POST/PUT/DELETE <code>/todo/lists</code>)	2
Todo CRUD (GET/POST <code>/todo/lists/:listId/todos</code> , PUT/DELETE <code>/todo/todos/:id</code>)	3
User profile (GET/PUT <code>/todo/users/:id</code>)	4
Todo dueDate on create/update	5

Authentication (Feature 1)

Method	Path	Auth	Purpose
POST	<code>/todo/register</code>	No	Create account
POST	<code>/todo/login</code>	No	Sign in; returns session payload
POST	<code>/todo/logout</code>	Yes	Invalidate session token

Register body:

```
{
  "fName": "Jane",
  "lName": "Doe",
  "email": "jdoe@example.com",
  "username": "jdoe",
  "password": "password123"
}
```

Login body:

```
{
  "username": "jdoe",
}
```

```
"password": "password123"
}
```

Register / login success (201 register · 200 login):

```
{
  "userId": 1,
  "username": "jdoe",
  "email": "jdoe@example.com",
  "fName": "Jane",
  "lName": "Doe",
  "role": "worker",
  "token": "<jwt>"
}
```

Logout success (200):

```
{
  "message": "Signed out successfully."
}
```

Common auth errors: missing fields 400 ; password < 8 chars 400 ; duplicate username/email 400 ; invalid login 401 with "Invalid username or password." ; missing/invalid token on protected routes 401 .

Lists (Feature 2)

Method	Path	Auth	Purpose
GET	/todo/lists	Yes	Lists owned by caller (array, ordered by name ASC)
POST	/todo/lists	Yes	Create a new list
PUT	/todo/lists/:listId	Yes	Rename a list
DELETE	/todo/lists/:listId	Yes	Delete a list owned by the caller

Create / rename body:

```
{ "name": "Groceries" }
```

List success (200 / 201):

```
{
  "id": 1,
  "name": "Groceries",
  "userId": 42,
  "createdAt": "2026-07-02T12:00:00.000Z",
  "updatedAt": "2026-07-02T12:00:00.000Z"
}
```

Delete success (200):

```
{ "message": "List deleted successfully." }
```

Validation errors: empty/whitespace name 400 with "List name is required." ; name > 100 chars 400 with "List name must be 100 characters or fewer." ; invalid listId 400 ; unowned list 404 with "List with id=<id> not found."

Todos (Feature 3)

Method	Path	Auth	Purpose
GET	/todo/lists/:listId/todos	Yes	Todos in an owned list (incomplete first, then createdAt ASC)
POST	/todo/lists/:listId/todos	Yes	Add a todo to an owned list
PUT	/todo/todos/:id	Yes	Update title and/or completed
DELETE	/todo/todos/:id	Yes	Delete a todo owned by the caller

Create body:

```
{
  "title": "Buy milk",
  "dueDate": "2026-07-15"
}
```

dueDate is optional. Omit it or send null for no due date.

Update body (any combination; omit a field to leave it unchanged except dueDate: null clears the date):

```
{
  "title": "Buy oat milk",
  "completed": true,
  "dueDate": "2026-07-20"
}
```

Clear due date:

```
{ "dueDate": null }
```

Todo success (200 / 201):

```
{
  "id": 10,
  "listId": 1,
  "title": "Buy milk",
  "completed": false,
  "dueDate": "2026-07-15",
  "userId": 42,
```

```
{
  "createdAt": "2026-07-02T12:05:00.000Z",
  "updatedAt": "2026-07-02T12:05:00.000Z"
}
```

`dueDate` is `null` when not set.

Delete success (`200`):

```
{ "message": "Todo deleted successfully." }
```

Validation errors: empty/whitespace title `400` with "Todo title is required." ; title > 255 chars `400` with "Todo title must be 255 characters or fewer." ; invalid `dueDate` `400` with "Due date must be a valid date in YYYY-MM-DD format." ; invalid ids `400` ; unowned list/todo `404` with "List with id=<id> not found." or "Todo with id=<id> not found."

User profile (Feature 4)

Method	Path	Auth	Purpose
GET	/todo/users/:id	Yes	Fetch caller's own profile (:id must equal req.user.id)
PUT	/todo/users/:id	Yes	Update caller's own profile

Update body:

```
{
  "fName": "Jane",
  "lName": "Doe",
  "email": "jane@example.com",
  "username": "jdoe",
  "password": "newpassword123"
}
```

`password` is optional. Omit it to leave the current password unchanged.

Profile success (`200`):

```
{
  "id": 42,
  "fName": "Jane",
  "lName": "Doe",
  "email": "jane@example.com",
  "username": "jdoe",
  "role": "worker",
  "createdAt": "2026-07-02T12:00:00.000Z",
  "updatedAt": "2026-07-02T12:05:00.000Z"
}
```

Password hash is never returned.

Validation errors: missing required fields 400 ; password < 8 chars 400 with "Password must be at least 8 characters." ; duplicate username 400 with "Username is already taken." ; duplicate email 400 with "Email is already registered." ; cross-user access 404 with "User with id=<id> not found."

behavior.md

Behavior & Rules Reference

Living snapshot of product rules currently in force on `dev` (not API shapes or columns — see [api.md](#) and [data-model.md](#)).

These files answer: *"What rules does the app enforce right now?"*

They do **not** authorize new scope — implement only from `features/feature-*.md` (**FR-00N** + Gherkin). Deep scenarios stay in the introducing feature; this file is an **index**.

Related: [ADR-0002 — Security architecture](#) (404 vs 403, ownership helpers)

Maintenance

When	Action
Feature changes a product rule (sort, ownership, validation, UI rule)	Update this file in the same PR
Feature only changes routes/payloads/schema	Update api.md / data-model.md ; touch this file only if rules changed
Drift suspected	Compare this file → code + mapped tests; fix reference or code

Auth & sessions

Rule	Enforcement	Introduced
Login is username + password (not email-only)	Auth API + Login UI	Feature 1
Passwords hashed with bcrypt (SALT_ROUNDS = 10); hash never returned	Register/login/profile APIs; user defaultScope	Features 1, 4
Session = JWT stored server-side; client sends Authorization: Bearer <token>	authenticate middleware + sessions table	Feature 1
Session lifetime 24 hours from creation	Session create on register/login	Feature 1
Login reuses a non-expired session for the same user when one exists	Login controller	Feature 1
Logout invalidates the server session and clears client user storage	Logout API + authService.logoutUser	Feature 1
Unauthenticated protected API → 401	authenticate	Feature 1
Unauthenticated protected UI → redirect to login	Router beforeEach	Feature 1
Default role for new users is worker; role is read-only in profile UI	Register + profile	Features 1, 4

Ownership & isolation

Rule	Enforcement	Introduced
Every authenticated request resolves to <code>req.user.id</code> from the session	<code>authenticate</code>	Feature 1
Cross-user access → 404 , never 403 (do not confirm existence)	<code>Controllers + getAccessibleOrNull</code>	ADR-0002; Features 2–4
Lists: reads/writes scoped to <code>userId = req.user.id</code> ; create ownership from server only	<code>list.controller + getAccessibleListOrNull</code>	Feature 2
Todos: parent list must be owned; todo reads/writes scoped to caller; create ignores client <code>userId/listId</code> spoofing	<code>todo.controller + helpers</code>	Feature 3
Profile: GET/PUT <code>/todo/users/:id</code> only when <code>:id === req.user.id</code>	<code>user.controller + getAccessibleUserOrNull</code>	Feature 4
Deleting a list cascades to its todos	<code>Sequelize List hasMany Todo onDelete: CASCADE</code>	Feature 3

Lists

Rule	Enforcement	Introduced
List name trimmed; empty/whitespace rejected	Create/update API + Dashboard dialogs	Feature 2
List name max 100 characters	API + client rules	Feature 2
Lists returned alphabetically by name	<code>findAll</code> order: name ASC	Feature 2
Single-view lists UI (<code>Dashboard.vue</code>); list CRUD via dialogs; no sidebar/main split	Dashboard	Feature 2
Empty lists: "No lists yet. Create your first list."	Dashboard	Feature 2

Todos

Rule	Enforcement	Introduced
Todo title trimmed; empty/whitespace rejected	Create/update API + dialogs	Feature 3
Todo title max 255 characters	API + client rules	Feature 3
New todos default completed: <code>false</code>	Create	Feature 3
Sort: incomplete first , then <code>createdAt</code> ascending	API order + client <code>sortTodos</code>	Feature 3
Items managed in list-items dialog (+ nested add/edit/delete); + Add Item only inside that dialog	Dashboard	Feature 3

Empty items: "No todos in this list yet."	Items dialog	Feature 3
Completed todos show struck-through / muted title	Dashboard row styling	Feature 3

Due dates

Rule	Enforcement	Introduced
dueDate optional on create/update; null = no due date	Todo API + dialogs	Feature 5
Calendar-only: API YYYY-MM-DD, DB DATEONLY	parseDueDateInput + model	Feature 5
Invalid due date string → 400 with due-date message	parseDueDateInput	Feature 5
PUT with dueDate: null clears; omitting dueDate leaves existing value	Update controller	Feature 5
Overdue styling only when incomplete and dueDate before today (browser local calendar)	isTodoOverdue + Dashboard	Feature 5
Completed past-due todos are not styled overdue	Same	Feature 5

Profile & MenuBar

Rule	Enforcement	Introduced
Profile fields trimmed; required strings rejected when empty	Profile PUT + Edit Profile dialog	Feature 4
Username normalized <code>trim().toLowerCase()</code> on save	User model hook + profile update	Features 1, 4
Password on profile update is optional; if set, min 8 chars and bcrypt hash	Profile PUT + dialog rules	Feature 4
Duplicate username → "Username is already taken."; duplicate email → "Email is already registered."	Profile PUT	Feature 4
Shared <code>emailRules</code> for register and Edit Profile	<code>frontend/src/config/validation.js</code>	Features 1, 4
After profile save: refresh <code>localStorage</code> user and dispatch <code>user-logged-in</code>	MenuBar	Feature 4
MenuBar: user icon → profile dropdown (name, username, email); Log out in dropdown only (no standalone Sign out)	MenuBar	Features 2→4
MenuBar hidden on login and register routes	<code>App.vue</code>	Feature 2

Errors (product convention)

Rule	Enforcement	Introduced
Error body shape { "message": "Human-readable explanation." }	Controllers	Features 1+

Validation / client spoof failures use 400 where specified; missing/unowned resources use 404

Controllers

Features 1–5

How to use

Question	Look here
What rule is in force now?	This file
Why was this rule chosen?	Feature FR / Gherkin, or ADR
Exact scenario / test name	Introducing feature-N-*.md Test Coverage Map
Routes and payloads	api.md
Tables and columns	data-model.md

data-model.md

Data Model Reference

Status: Integrated schema through **Feature 5** (`users` , `sessions` , `lists` , `todos` with optional `dueDate`).

Authority for new work: feature specs in `features/` — update this file in the same PR when schema changes.

Architecture: [ADR-0003 — MySQL relational database](#)

Feature provenance

Table / column	Introduced
<code>users</code> , <code>sessions</code>	Feature 1
<code>users</code> profile GET/PUT	Feature 4 (same table)
<code>lists</code> (CRUD)	Feature 2
<code>todos</code>	Feature 3
<code>todos.dueDate</code>	Feature 5

users

Column	Type	Rules
<code>id</code>	INTEGER PK	Auto-increment
<code>fName</code>	STRING	Required
<code>lName</code>	STRING	Required
<code>email</code>	STRING	Required, unique
<code>username</code>	STRING(100)	Required, unique; trimmed and stored lowercase (<code>beforeValidate</code> hook)
<code>password</code>	STRING(255)	Required; bcrypt hash only (never returned by API)
<code>role</code>	STRING(20)	Default <code>worker</code>
<code>createdAt</code>	DATE	Sequelize timestamps
<code>updatedAt</code>	DATE	Sequelize timestamps

Sequelize: `defaultScope` excludes `password` from query results. Use `unscoped()` when comparing passwords at login or updating password on profile `PUT` .

sessions

Column	Type	Rules
--------	------	-------

id	INTEGER PK	Auto-increment
token	STRING	Required; JWT string; cleared to "" on logout
email	STRING	Required
expirationDate	DATE	Required; 24-hour lifetime from creation
userId	INTEGER FK	Required → users.id
createdAt	DATE	Sequelize timestamps
updatedAt	DATE	Sequelize timestamps

lists

Column	Type	Rules
id	INTEGER PK	Auto-increment
name	STRING(100)	Required
userId	INTEGER FK	Required → users.id
createdAt	DATE	Sequelize timestamps
updatedAt	DATE	Sequelize timestamps

Note: dueDate is optional (DATEONLY , nullable).

todos

Column	Type	Rules
id	INTEGER PK	Auto-increment
listId	INTEGER FK	Required → lists.id; cascade on list delete
title	STRING(255)	Required
completed	BOOLEAN	Default false
userId	INTEGER FK	Required → users.id
dueDate	DATEONLY	Nullable; optional on create/update (Feature 5)
createdAt	DATE	Sequelize timestamps
updatedAt	DATE	Sequelize timestamps

Associations

- User hasMany Session — onDelete: CASCADE
- Session belongsTo User

- User hasMany List — onDelete: CASCADE
- List belongsTo User
- List hasMany Todo — onDelete: CASCADE
- Todo belongsTo List
- User hasMany Todo — onDelete: CASCADE
- Todo belongsTo User

writing-living-reference.md

Writing Living Reference

A student guide for maintaining `features/reference/` — the **current integrated snapshot** of the product on `dev` .

Index: [README.md](#)

Files: [api.md](#) · [data-model.md](#) · [behavior.md](#)

Related: [framework.md](#) (merge / DoD) · [writing feature design](#) · [writing feature requirements](#)

What living reference is

Living reference answers: “*What does the app look like / enforce right now on `dev` ?*”

Artifact	Answers	Authorizes new work?
Feature spec (<code>feature-N-*.md</code>)	<i>What changes this feature adds</i>	Yes — implement from here
Living reference (<code>features/reference/</code>)	<i>What exists after merges</i>	No — snapshot only
ADRs	<i>Why cross-cutting architecture</i>	Indirectly (constraints)
Cursor rules	<i>How to code</i>	Patterns, not product scope

Feature specs are **deltas**. Reference files are **current state**. After Feature 5 ships, `api.md` shows the full `/todo/` surface; Feature 5’s spec only describes the due-date delta.

Reference is **not** auto-generated from specs. Update it in the **same PR** as the implementation (required Definition of Done).

The three files

File	Contents	Update when...
api.md	Routes, auth, request/response shapes, errors	Routes or payloads change
data-model.md	Tables, columns, types/rules, associations	Schema or associations change
behavior.md	Product rules in force (ownership, sort, validation, UI rules)	Rules change (not mere path renames)

Which file(s) for this change?

You changed...	Update
New/changed endpoint or JSON fields	api.md
New table/column/FK/association	data-model.md
Ownership, sort order, validation limits, empty-state copy, overdue rules, where Log out lives	behavior.md

Only internal refactor, same external contract	Usually none (confirm no behavior change)
Screen layout polish already implied by existing rules	Usually none ; if a stated UI rule changes → behavior.md

Many features touch **two or three** files (e.g. Feature 5: `dueDate` column → data-model + api + overdue rule → behavior).

Principles for writing living reference

1. **Describe now, not history.** Reference is the integrated product. Provenance tables record *which feature introduced* an area — not a full changelog essay.
2. **Never authorize scope.** If someone needs new behavior, they write/update a **feature spec** first. Do not “fix production” by editing only `api.md`.
3. **Update in the same PR as code.** DoD — not a follow-up “docs later” task.
4. **Edit as a delta on current state.** Add a section, field, or row; don’t rewrite the whole file unless structure is broken.
5. **Match shipped code.** If the PR returns `dueDate` as `YYYY-MM-DD`, the reference must say that — not an older draft from the spec that never shipped.
6. **Keep Gherkin in features.** Reference may index rules; deep scenarios stay in `feature-N-*.md` Acceptance Criteria.
7. **One fact, one place.** Column types live in **data-model**; HTTP paths in **api**; “lists sort A–Z” in **behavior** — don’t triple-paste novels.
8. **Record provenance.** When you add an area, add/update the Feature provenance table (Introduced / Feature N).
9. **Align with Accepted ADRs.** Ownership/ `404` rules should match ADR-0002; don’t invent a conflicting “use 403” in `behavior.md`.
10. **Stubs stay empty until Feature 1+.** New apps / `reset:example` start with empty reference shells; fill as features merge.

Writing api.md

What belongs

- Base path and auth convention (`Authorization: Bearer ...`)
- Endpoint tables: Method, Path, Auth, Purpose
- Request/response JSON for create/update and important errors
- Feature provenance for API areas

Principles

Do	Don’t
Document the integrated API after this merge	Only paste this feature’s API section and delete older routes
Show real status codes and error { <code>"message": ...</code> } shapes	Vague “returns an error”
Note Auth Yes/No per route	Assume readers remember Feature 1
Update payloads when fields are added (e.g. <code>dueDate</code>)	Leave Feature 3 payloads forever when Feature 5 shipped

Example delta (Feature 5 style)

- Keep existing todo routes.
- Extend create/update body and response docs with optional `dueDate`.
- Add provenance row: Todo `dueDate` → Feature 5.

Writing data-model.md

What belongs

- Each table: Field / Type / Rules
- Associations (User hasMany List , cascade deletes, ...)
- Feature provenance for schema areas

Principles

Do	Don't
Reflect shipped Sequelize/MySQL reality	Speculative columns "we might need"
Mark PK, FK, unique, defaults, DATEONLY vs DATETIME	Copy-paste model .js source
For deltas, add/change only the new fields and note the feature	Delete unrelated tables from the file
Keep associations in sync with models/index.js	Orphan "hasMany" that code doesn't implement

Writing behavior.md

What belongs

Product **rules** currently enforced — ownership, sorting, validation limits, session lifetime, empty-state **copy**, overdue styling rules, MenuBar/logout placement.

Use a compact table shape:

Rule	Enforcement	Introduced
Cross-user access → 404 , never 403	Controllers + helpers	ADR-0002; Features 2–4
Lists returned alphabetically by name	findAll order	Feature 2

Principles

Do	Don't
State the rule in product language	Dump full Gherkin scenarios
Point at enforcement (middleware, helper, UI)	"The app handles this somehow"
Add a row when a rule changes	Duplicate every API path from api.md
Group by area (Auth, Ownership, Lists, Todos, ...)	One undifferentiated bullet blob

behavior.md vs Screen Requirements: Screen Requirements in the feature authorize UI for that slice. After merge, durable UI **rules** (exact empty-state string, overdue condition) that others must not regress belong in **behavior.md**.

Workflow (per feature PR)

1. Implement from `features/feature-N-*.md` only.
2. Before marking DoD complete, ask:

- Schema change? → **data-model.md**
- Route/payload change? → **api.md**
- Rule change? → **behavior.md**

3. Update provenance tables.

4. Skim for contradictions (api field missing from data-model; behavior sort order ≠ api order).

5. List the files you touched in the feature's **Agent implementation request** / DoD checkboxes.

Agent implementation request (reminder)

Features should say which reference files to update, for example:

If API routes, payloads, schema, or product rules changed per this spec, update @features/reference/api.md, @features/reference/data-model.md, and/or @features/reference/behavior.md in the same PR to match shipped code.

Reference updates for this feature: list only the files that will change (or none).

Provenance tables

Keep a short “who introduced this” index in README and/or each file:

Area	Introduced
Auth, sessions	Feature 1
Lists CRUD + Dashboard lists view	Feature 2
...	...

Principles: one row per capability area; update when a **new** area appears; for field-level deltas, a row like Todo dueDate → Feature 5 is enough.

Drift and repair

Symptom	Action
Code has a route not in api.md	Update api.md (if intentional) or remove the route (if unauthorized)
Spec says one thing, reference another, code a third	Spec authorizes; fix code + reference to match the shipped spec delta
behavior.md missing a rule that tests enforce	Add the rule row
Tempted to add behavior only in reference	Stop — write a feature delta first

Checklist (before merge)

- ☐ Knew which of api / data-model / behavior apply (or consciously chose none)
- ☐ Edits describe **current** integrated state after this PR
- ☐ No new scope that isn't in the feature spec
- ☐ Provenance updated for new areas/fields

- ☐ api ↔ data-model ↔ behavior consistent with each other and with tests
- ☐ Feature DoD / Agent request lists the reference files touched
- ☐ Did not “clean up” by deleting still-shipped older feature surface area

Anti-patterns

Avoid	Do instead
Updating reference weeks after merge	Same PR as implementation
Replacing api.md with only this feature’s endpoints	Merge delta into the full snapshot
Using reference as the requirements doc	Feature specs authorize; reference reflects
Copying entire feature Data Model / API sections verbatim forever	Integrate into current-state structure; drop redundancy
behavior.md as a second Gherkin suite	Rule index + enforcement pointer
Editing reference to “make the agent happier” without a spec	Spec first (constitution)

How this fits the other guides

Author the delta (what/how to build) → writing-feature-requirements.md
writing-feature-design.md

Ship the delta + snapshot “what is” → writing-living-reference.md (this file)

Architecture why → docs/adr/writing-adrs.md

Quality bars → docs/nfr/writing-quality-attributes.md

After merge, the next feature’s authors read **reference** to know the baseline and write a **new feature file** for the next delta — they do not rewrite Features 1–N.